

NCHU - [DRL][FR] Final Proposal Phase 4

media.mp4

檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

Frank_oral_defense_0725_v7.pptx

簡報

我想要開發一個 Deep Reinforcement Learning 專案

如附圖 1 ilearning 平台上的描述

但我現在還沒找到要做的主題 請你建議我

請你在完理解過教授要的 proposal 架構後再推薦我最適合用教授這套 A-to-Z 的論文寫作方式做 **final project proposal**

附檔是教授提供的 reference 請幫我完整全部檔案都看完, 理解後

推薦我最適合套用進此流程的 DRL 題目

附上教授 A-to-Z 的論文寫作流程影片介紹:

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

顯示更多顯示較少

我看完了你貼的 ilearning 作業說明，也把三個附檔都過了一遍。我的判斷很明確：

最適合你拿來做這門 DRL final project proposal 的題目，是：

「用 Deep Reinforcement Learning 做都市路口自適應交通號誌控制（Adaptive Traffic Signal Control）」。

你附的兩份 drone reference，其實是在示範同一種教授喜歡的敘事方式：先把問題的重要性講大，再整理 related work，接著把方法流程、資料/系統設計、實驗比較、限制與未來導入講清楚。口試簡報版明確就是 **Motivation → Related Works → Proposed Method → Experimental Results → Conclusion**，還把資料量、模型、限制都量化寫出來；Smart City PDF 則把它壓成「問題—系統流程—實驗—風險—落地路線圖」的簡報故事。至於 2330 那份雖然不是 DRL 論文，但它很像一個「固定頁面模板 + 關鍵結論 + 證據框 + 決策邏輯」的視覺報告範本，這對你做 proposal 投影片其實很有參考價值。

Frank_oral_defense_0725_v7 Smart_City_Drone_Law_Enforcement

2330_SMC_FVG_MTF_Report_A4_Port...

再加上你附的影片頁面就是 **Project Proposal A to Z (字幕外加)**；搭配你上傳的 mp4 內容，我理解教授現在要的不是「先把系統全部做完」，而是要你先把整條論文故事線講順：**這個問題為什麼值得做、現在卡在哪裡、你打算怎麼設計、未來要怎麼驗證**。影片裡對 abstract 的要求其實很清楚，就是 **Attention Getter / Motivation → But → Cure → Development → Experiments → Findings**；而實驗段落也要先想好 **baseline、metrics、setup、tuning、useful results**。  YouTube

為什麼我最推薦「DRL 自適應交通號誌控制」

第一，它最貼近教授 **reference** 的味道。你手上的 reference 是智慧城市、無人機、執法、系統導入，核心共同點不是「影像辨識」而已，而是**真實城市問題 + AI/系統化設計 + 可驗證的實驗**。交通號誌控制同樣是很典型的智慧城市議題，問題好講、價值大、實驗也能量化。

第二，它非常適合 **A-to-Z** 的 **proposal** 寫法。

這題天然就能寫成：

- Attention Getter / Motivation：都市壅塞、固定時制浪費通行效率
- But：固定式或傳統 rule-based 控制難以適應動態車流
- Cure：用 DRL 根據即時狀態決定號誌切換
- Development：定義 state / action / reward，建立 SUMO 模擬環境
- Experiments：和 fixed-time、actuated control、DQN/PPO 比較
- Findings：看平均等待時間、queue length、travel time 是否下降

第三，它**工具鏈成熟、可重現、風險低**。SUMO 是官方維護的開源微觀交通模擬器，能處理大型路網；SUMO-RL 已經把 Traffic Signal Control 包成可直接拿來做 RL 的環境，而且支援 Gymnasium、Stable-Baselines3、RLlib，也能改 state 和 reward；Gymnasium 本身提供標準 RL API 與自訂環境教學；Stable-Baselines3 則提供穩定的 PyTorch 演算法實作，像 PPO 與 DQN 都可直接用。🌱 [Eclipse SUMO+7](#)

第四，它**很容易控制專案範圍**。你可以先做「單一路口」拿到能跑的結果，再決定要不要擴成「多路口」或「multi-agent」。SUMO-RL 也已經明講支援 multi-agent；PettingZoo 則是 Farama 官方的多智能體 RL 介面標準。也就是說，這題從保守版到進階版都有路可以走。📄 [Lucas Alegre+1](#)

我會怎麼幫你定這題的正式題目

我建議你不要只寫成很泛的「Traffic Signal Control using DRL」，而是寫成：

中文題目

以需求感知深度強化學習進行都市路口自適應交通號誌控制

英文題目

Demand-Aware Deep Reinforcement Learning for Adaptive Traffic Signal Control at Urban Intersections

這樣的好處是你 proposal 裡的「novelty」比較容易站住腳：

你不是只說「我用 DRL」，而是說你要做 **demand-aware state design** 或 **reward shaping**，讓 agent 不只顧單一方面，而是同時兼顧等待時間、排隊長度與通行效率。

這題最適合的 proposal 骨架

1. Introduction / A-to-F

先講城市壅塞與固定號誌的侷限，再指出現有方法在動態需求、尖離峰切換、不同流向公平性上的不足，最後收斂到你的研究問題：

「能不能讓 DRL 根據即時車流自適應調整號誌，相比固定時制顯著降低等待成本？」

2. Related Work / NotebookLM

文獻建議分三條線整理：

- 傳統交通號誌控制：fixed-time、actuated、rule-based
- RL / DRL traffic signal control：Q-learning、DQN、PPO、multi-agent
- reward / state design：queue、delay、pressure、fairness

教授 reference 不是單純列 paper，而是有「從舊到新、從簡單到複雜」的演進感，這點你要學。

Frank_oral_defense_0725_v7

3. Proposed Design / 規格書

這一段你要寫得像系統設計，而不是只有概念：

- Environment：SUMO / SUMO-RL
- Agent：先用 DQN 或 PPO
- State：各車道 queue length、waiting time、current phase、phase elapsed time
- Action：切換到哪個 phase，或維持 / 切換
- Reward：負的總等待時間 + queue penalty
- Baselines：fixed-time、actuated control
- Metrics：average waiting time、queue length、throughput、travel time

我建議你的最小可行版本

不要一開始就做太大。最穩的版本是：

- 場景：單一路口
- 演算法：DQN、PPO 二選一，外加 fixed-time baseline
- 目標：讓 average waiting time 明顯下降
- 可選加分點：再加入第二個 reward term，處理 fairness 或尖峰流量

這樣 proposal 好寫，final 也比較做得完。

我不推薦你現在優先選的題目

1. 無人機巡航路徑規劃 DRL

題目很帥，也跟 reference 很像，但實作風險高。AirSim 官方雖然有 RL 範例，但文件自己也說這一塊仍在 active development；你很容易把時間花在模擬器與飛行控制，而不是論文主體。  Microsoft GitHub

2. CARLA 自動駕駛 DRL

CARLA 本身很強，是為自駕研究打造的開源模擬平台；但官方 RL repo 也明寫了只放 inference code、training code 沒有釋出。這種題目工程量通常比課程 final project 大很多。  CARLA Simulator+1

三個候選題目的結論比較

題目	和教授 reference 的契合度	Proposal 好寫程度	實作風險	我建議
DRL 自適應交通號誌控制	9.5/10	9.5/10	4/10	最推薦
DRL 無人機巡航 / 巡檢路徑規劃	8.5/10	7/10	7.5/10	第二順位
DRL 自動駕駛 (CARLA)	7/10	6/10	9/10	不建議當首選

我的最終建議

你的情況下，最穩、最像教授會買單、最能套進 A-to-Z 流程的，就是「DRL 自適應交通號誌控制」。

因為它同時滿足四件事：

- 有**真實世界動機**
- 有**明確 baseline 與 metrics**
- 有**成熟工具鏈**
- 有**好寫的論文故事**

你決定選這題的話，我下一則就直接幫你寫成**教授格式的 final project proposal 初稿**，包含：
題目、abstract、introduction、related work 架構、proposed design、experiment plan、預期貢獻。

<https://www.youtube.com/shorts/qeNim1T082Y>

這個盲人眼鏡開源專案 可以嗎
也幫我把 github 位置找出來

可以，但要分開看：

如果你是問「**這個題目能不能當 final project proposal**」：可以。

如果你是問「**可不可以直接當 DRL 專案原封不動拿去做**」：我不建議。

你貼的這支 Shorts 看起來對應的是近來流傳的那個「95 後手搓 AI 導盲眼鏡」開源專案；相關文章直接指向同一個 GitHub 倉庫，而該 repo 自己也寫明這是一個面向視障人士的智能導航與輔助系統。 [YouTube+2](#)

這個專案本身其實**很完整，也很有 proposal 故事性**：它整合了盲道導航、過馬路輔助、物品識別查找、即時語音互動；技術上用到 YOLO 分割做盲道辨識、YOLO-E + ByteTrack 做目標追蹤、MediaPipe 做手部引導，並結合 Qwen-Omni 與阿里雲 DashScope 做語音與多模態互動。repo 也標示為 MIT license。 [GitHub+1](#)

但如果你的課是 **Deep Reinforcement Learning**，那這個 repo **目前不是一個 DRL 專案**。我查它的公開 README，找不到 reinforcement learning、RL、DQN、PPO 之類的設計痕跡；從結構看起來，它更像是**電腦視覺 + 多模態語音 + 邊緣裝置整合**的 assistive AI 系統，而不是用 agent 學 policy 的 DRL 研究。 [GitHub+3](#)

所以我的判斷是：

結論一：可以當靈感來源，但不要直接把它當成「DRL 題目」交上去。

因為教授要的是可以用 A-to-Z 論文方式講清楚的 proposal，而這個專案原型雖然很吸睛，卻比較偏系統整合。若你直接照搬，老師很可能會問：「你的 reinforcement learning 在哪裡？」 [GitHub+1](#)

結論二：最好的做法，是把它改寫成一個“以這個導盲眼鏡為應用場景”的 DRL 題目。

例如你可以改成：

- **DRL-based Assistive Navigation Policy for Blind-Path Following and Obstacle Avoidance**
- **Using Deep Reinforcement Learning to Optimize Audio Guidance for Smart Glasses Navigation**
- **A DRL Decision Module for Crosswalk Assistance in Smart Glasses for the Visually Impaired**

這樣就能把原本的 CV 感知模組保留成「環境輸入」，而把真正的研究貢獻放在 **agent 如何根據狀態決定下一步提示/方向/避障策略**。這樣比較符合 DRL 課程定位，也更容易寫成教授喜歡的 A-to-Z 結構。

GitHub 位置

倉庫是：[AI-FanGe/OpenAIGlasses_for_Navigation](#)。文章與 repo 互相對得上，repo 首頁也明確寫它是「a open framework for blind navigation based on esp32」。[知乎專欄+1](#)

另外提醒你兩個現實面問題。第一，repo 自己明寫**僅供交流學習，不要直接給視障人群使用**，所以你 proposal 最好定位成研究原型，不要寫成 ready-to-deploy 產品。第二，它的 README 顯示需要 GPU、CUDA、ESP32-CAM、DashScope API key，而且 quick start 裡的 clone 指令還留著像 placeholder 的路徑，代表專案雖然開源，但工程細節還有不少要自己補。[GitHub+2](#)

我的建議會是：

這題可以做，但要改成「導盲眼鏡中的 DRL 決策/導航策略」而不是「開源導盲眼鏡系統重現」。

你若要，我下一步可以直接幫你這個題目改寫成一份符合教授 A-to-Z 架構的 **proposal 題目 + Abstract + Introduction + Related Work + Proposed Design**。

貼上的 Markdown (1).md

檔案

好 幫我詳細推薦跟 "網路吞吐量設計應用 DRL" 有關的 能獲得 100 分滿分的期末報告題目
請推薦我 10 個題目並作比較分析
最後推薦你心目中的第一名給我

搜尋前請先看我先前給你的 Final Project Proposal 要求 如附圖所示 我要能適合先提案再收斂成期末專題
且能更做出來的題目

media.mp4

檔案

Smart_City_Drone_Law_Enforcement.pdf

PDF

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Frank_oral_defense_0725_v7.pdf

PDF

From 0 to 1 Deep RL with SMC MDP.pdf

PDF

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

附上三種參考資料(請自行閱讀附檔後自動對應以下範圍) 幫我做成同樣的研究流程

1. A to Z 論文撰寫框架: 有 2 份
2. 專題提案成果: 有 3 份
3. 教授的 chatgpt 收斂出專題提案的對話過程: 有 1 份

我的要求目前只有以下三點 其餘需求請你幫我補充成可以獲得 100 分滿分期末專題提案的程度交付物 並幫我規劃推進步驟:

1. 讓 "以深度強化學習進行單一瓶颈鏈路壅塞控制與吞吐量最佳化" 這題目能產出如 "專題提案成果" 的最終交付物給我
 2. 我的所有程式碼工作都會交給 `antigravity` 執行工作且要用到工具鏈 `**"openspec"**`
 3. final project 都會交給 ai agent `antigravity` 執行 我只負責規劃及下指令 部參與實際程式開發 我開發的是整個專案 這是我的角色
- 顯示更多顯示較少

Phase 2/OpenSpec change 04 : `drl-mvp` 階段驗收沒問題

****那我們進入下一個階段****

請用一樣的生成標準（最完美且超高品質）產出「Phase 3：Baseline 先行」的****最佳 prompt****

以下是 **Phase 3：Baseline 先行** 的「最完美且超高品質」最佳 prompt。

這一階段開始從 Phase 2 的 OpenSpec 文件進入**可執行準備與 baseline 實作指揮**，但核心仍不是直接衝 DQN，而是先讓 Antigravity 依照 **change-02-baseline-benchmark** 建立 ns-3 單一瓶颈鏈路 baseline。ns-3 官方 TCP model 支援 NewReno、CUBIC、BBR 等 congestion control 演算法；Flow Monitor 可提供 flow-level 統計，例如 tx/rx bytes、packets、lost packets、delaySum 等；ns-3 tracing 也被官方教學定位為從 simulation 取出資訊的重要機制。這些會被要求寫入 Phase 3 的執行規格與驗收條件。^{NSNAM+2}

同時，OpenSpec 的目的就是讓人與 AI coding assistant 在寫 code 前先對齊規格，並以 change folder 集中管理 proposal、design、tasks、specs；所以 Phase 3 必須明確要求 Antigravity **先讀已驗收的 change-01~04，再只執行 baseline，不可跳到 ns3-gym / DQN / PPO**。^{GitHub+1}

請下一回合直接貼上以下整段：

你現在要執行的是我 DRL Final Project 的下一階段：

Phase 3:Baseline 先行

最高任務目標

你要基於已驗收完成的 Phase 2 OpenSpec 四個 change：

- change-01-project-charter
- change-02-baseline-benchmark
- change-03-opengym-env
- change-04-drl-mvp

替我產出一份可以交給 ****Antigravity**** 執行的 ****Phase 3:Baseline 先行實作指揮包****。

注意：這一階段不是訓練 DQN，不是寫 ns3-gym environment，不是進入 DRL。
這一階段的唯一目標是：

> 先把 ns-3 single bottleneck baseline benchmark 做出來，並產出可支撐後續 DQN comparison 的 logs、tables、figures 與 README 說明。

你要把 Phase 2 的 OpenSpec 規格轉成 Phase 3 的「執行指令、任務拆解、驗收條件、風險降階、Antigravity

prompt」。

一、已完成前置階段

1. Phase 2 已驗收完成

以下 change 已通過：

`01-project-charter`

已凍結：

- 題目
- scope / non-goals
- toolchain
- baseline list
- metrics
- MVP definition
- downstream change map

`02-baseline-benchmark`

已凍結：

- NewReno / CUBIC / BBR baseline roles
- single bottleneck topology concept
- throughput / RTT or delay / loss / utility metrics
- scenario matrix
- output artifacts
- benchmark evaluation philosophy
- baseline-first 原則

`03-opengym-env`

已凍結：

- MDP interface
- environment boundary
- observation space
- discrete action space
- reward philosophy
- reset / step conceptual flow
- info dictionary
- random agent smoke test specification

`04-drl-mvp`

已凍結：

- DQN MVP boundary
- training protocol concept
- evaluation protocol concept
- baseline comparison protocol
- output artifacts
- success / failure criteria
- extension rules

Phase 3 必須完全承接上述內容，不得自行改題、擴題、跳到 DQN、跳到 PPO，或直接做 IPFS / QUIC / multi-agent / multi-path。

2. 正式題目

中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

GitHub repo name

drl-bottleneck-congestion-control

二、Phase 3 的核心目的

Phase 3 的任務是建立真正可用的 **ns-3 baseline benchmark foundation**。

它要回答：

1. ns-3 baseline benchmark 應如何啟動？
2. single bottleneck topology 的 MVP 實作順序應如何安排？
3. NewReno / CUBIC / BBR 應如何分層執行？
4. 若 BBR 有風險，如何不阻塞 MVP？
5. throughput / RTT or delay / loss / utility 要如何被記錄與輸出？
6. raw logs、summary CSV、plots、metadata 要如何組織？
7. S1 / S2 / S3 / S4 scenario 應如何分層執行？
8. Phase 3 哪些成果可以直接支撐 Phase 4 DQN MVP？
9. 如果 baseline 結果異常，應如何 debug 與降階？
10. 要給 Antigravity 哪一段明確、可執行、不偏航的 implementation prompt？

三、本階段允許與禁止事項

3.1 Phase 3 允許做什麼

Phase 3 允許規劃並交給 Antigravity 執行：

- 建立 ns-3 baseline benchmark 專案骨架
- 建立 single bottleneck topology
- 執行 NewReno baseline
- 執行 CUBIC baseline
- 嘗試執行 BBR baseline
- 若 BBR 有版本或設定風險，降階為 strongly recommended / optional
- 建立 baseline logging pipeline
- 匯出 raw logs
- 匯出 summary CSV
- 匯出 per-scenario summary table
- 產出 throughput plot
- 產出 RTT / delay plot
- 產出 packet loss plot
- 產出 utility comparison plot
- 建立 experiment metadata
- 更新 README 的 baseline 章節
- 產出 Phase 3 驗收報告

3.2 Phase 3 禁止做什麼

Phase 3 嚴格禁止：

- 不建立 ns3-gym environment
- 不寫 DQN training script
- 不啟動 DQN training
- 不接 PPO
- 不做 reward ablation
- 不做 observation ablation
- 不接 IPFS
- 不接 QUIC
- 不修改 Linux kernel TCP
- 不做 multi-agent RL
- 不做 multi-path routing

- 不做大型 topology
- 不把 Pantheon 設為必裝工具
- 不宣稱 DRL 結果
- 不把 baseline 結果硬解讀成 DRL 成果
- 不在未完成 baseline 前進入 Phase 4

四、你必須先搜尋並引用的來源

請先做最新網路搜尋，不可只靠記憶。

優先使用來源

請優先搜尋並引用：

1. OpenSpec 官方網站 / GitHub README / docs
2. OpenSpec concepts / workflow 文件
3. ns-3 官方 TCP model 文件
4. ns-3 Flow Monitor 官方文件
5. ns-3 tracing / tutorial / statistics 官方文件
6. ns-3 官方 examples / tutorial, 如有需要
7. ns3-gym 官方 GitHub / ns-3 App Store 頁面
8. Aurora 原始論文
9. Pantheon paper / benchmark 說明
10. 必要時補充正式論文或 RFC

你必須用來源確認的判斷

請確認並支撐以下判斷：

- ns-3 適合做 TCP / congestion control baseline
- ns-3 支援 NewReno、CUBIC、BBR 等 TCP congestion control
- Flow Monitor 可支援 flow-level statistics, 適合 throughput / delay / loss 類量測
- ns-3 tracing 可用於更細緻的 simulation data extraction
- baseline-first 是後續 DQN comparison 的必要前置
- Pantheon 可作 benchmark / reproducibility 思想來源，但不是 Phase 3 必裝工具
- Phase 3 應輸出可重現、可追蹤、可視覺化的 baseline artifacts
- 如果 BBR 或 RTT logging 不穩，應採降階策略而非卡死整個專案

五、這次必須輸出的內容格式

請嚴格按照以下章節輸出，不要跳章，不要偷省略。

Phase 3: Baseline 先行總結

請用 150–250 字說明：

- Phase 3 的目的
- 它如何承接 Phase 2 四個 OpenSpec changes
- 為什麼 baseline 必須先於 DQN
- Phase 3 的最小可行成果是什麼
- Phase 3 如何避免 Antigravity 直接跳到 DRL

1. Phase 3 Mission / Boundary

請輸出：

1.1 Phase 3 Mission

請正式說明 Phase 3 的使命。

1.2 Phase 3 In Scope

列出可做事項。

1.3 Phase 3 Out of Scope

列出禁止事項。

1.4 Phase 3 Success Definition

請定義：

成功不等於什麼。

成功等於什麼。

2. Phase 3 Repo / Directory Blueprint

請設計 Phase 3 建議 repo 結構。

至少包含：

```
```text
drl-bottleneck-congestion-control/
 README.md
 openspec/
 src/
 ns3/
 scratch_or_examples/
 helpers/
 experiments/
 configs/
 raw_logs/
 summaries/
 metadata/
 figures/
 baseline/
 reports/
 phase3-baseline/
 scripts/
 analysis/
```

請對每個資料夾說明：

- purpose
- should contain
- should not contain
- acceptance criteria

注意：

這是 repo blueprint，不是實際建立檔案。

## # 3. Baseline Implementation Roadmap

請把 Phase 3 拆成清楚的 implementation roadmap。

至少包含以下階段：

### 3.1 Step A : Environment / Toolchain Verification

目的：

- 確認 ns-3 版本
- 確認 TCP variants 可用
- 確認 Flow Monitor 或 tracing 可用
- 確認 repo 結構

輸出：

- toolchain metadata
- version notes
- risk notes

### 3.2 Step B : Minimal Single Bottleneck Topology

目的：

- 建立 one sender / one receiver / one bottleneck link
- 建立 one long-lived TCP flow
- 不做 DRL、不做 ns3-gym

輸出：

- topology description
- topology metadata
- sanity check note

### 3.3 Step C : NewReno Baseline

目的：

- 先跑最基礎 TCP baseline
- 驗證 logging pipeline

輸出：

- NewReno raw logs
- NewReno summary
- metrics sanity check

### 3.4 Step D : CUBIC Baseline

目的：

- 跑主要 modern TCP baseline
- 和 NewReno 使用同一 metrics 口徑

輸出：

- CUBIC raw logs
- CUBIC summary
- NewReno vs CUBIC comparison table

### 3.5 Step E : BBR Baseline

目的：

- 嘗試 BBR
- 若 BBR 有問題，不阻塞 MVP

輸出：

- BBR result or BBR fallback note
- version / availability note

### 3.6 Step F : Scenario Matrix Execution

目的：

- 先完成 S1 / S2
- 再視情況做 S3
- S4 僅 optional

輸出：

- per-scenario baseline summaries
- scenario metadata

### 3.7 Step G : Visualization and Reporting

目的：

- 產出可放入 PPT / README 的 baseline figures

輸出：

- throughput plot
- RTT / delay plot
- packet loss plot
- utility plot
- baseline summary table

- Phase 3 report
- 

## # 4. Scenario Execution Plan

請根據 change-02 的 scenario matrix 設計 Phase 3 執行順序。

必須包含：

### 4.1 MVP-required Scenarios

- S1 Stable low-delay bottleneck
- S2 Stable high-delay bottleneck

### 4.2 Should-have Scenario

- S3 Variable bandwidth bottleneck

### 4.3 Optional Scenario

- S4 Bottleneck with cross traffic

請為每個 scenario 提供：

- priority
  - purpose
  - required baselines
  - required metrics
  - expected artifact
  - risk
  - fallback
- 

## # 5. Metrics / Logging / Artifact Requirements

請定義 Phase 3 必須產出的 metrics 與 artifacts。

### 5.1 Metrics

至少包含：

- average throughput
- average RTT / delay
- packet loss rate
- utility score

## 5.2 Raw Logs

請定義 raw logs 應包含：

- scenario ID
- method
- run ID
- seed if applicable
- received bytes
- tx / rx packets
- delay / RTT related fields
- lost / dropped packets if available
- simulation duration
- metadata

## 5.3 Summary CSV

請定義 summary CSV 欄位。

至少包含：

- scenario\_id
- method
- run\_id
- throughput
- avg\_delay\_or\_rtt
- loss\_rate
- utility
- notes

## 5.4 Figures

至少包含：

- throughput comparison
- delay / RTT comparison
- loss comparison
- utility comparison

## 5.5 README / PPT-ready Summary

請定義後續 README / PPT 應如何使用 baseline 結果。

## # 6. Utility Score Specification

請定義 Phase 3 utility score 的執行口徑。

必須包含：

- 為什麼需要 utility
- 為什麼不能只看 throughput
- conceptual formula
- normalization requirement
- weights not final / can be revised
- 如何避免誤讀

請使用：

$\text{utility} = \text{normalized\_throughput} - \text{penalty\_delay} - \text{penalty\_loss}$

請提供：

- 符號注釋
- 白話說明
- 注意：若 Phase 3 不足以穩定定義 utility，可先輸出 raw metrics，utility 作 provisional / analysis metric

---

## # 7. Acceptance Criteria

請建立 Phase 3 驗收條件。

至少包含：

### 7.1 Minimum Acceptance

例如：

- repo structure ready
- S1 / S2 completed for NewReno and CUBIC
- raw logs available
- summary CSV available
- at least throughput and delay / RTT comparison available
- BBR fallback documented if unavailable

### 7.2 Strong Acceptance

例如：

- BBR completed



- S3 completed
- loss and utility plots available
- metadata complete
- README baseline section completed

### 7.3 Excellent Acceptance

例如：

- S4 completed
- repeat runs / seeds
- all figures PPT-ready
- limitation notes complete
- artifacts cleanly organized

## # 8. Risk Register and Fallback Strategy

請建立 Phase 3 專屬風險清單。

至少包含：

1. ns-3 安裝 / 版本問題
2. TCP variant 名稱或 API 不一致
3. BBR 不可用
4. Flow Monitor 無法取得 RTT
5. delay / RTT 口徑混淆
6. loss rate 計算不穩
7. throughput 計算時間窗不一致
8. scenario 太複雜
9. S3 / S4 失控
10. baseline 結果異常
11. 圖表無法支撐 PPT
12. Antigravity 直接跳到 DQN
13. repo artifacts 混亂
14. metadata 缺失

表格欄位請包含：

- Risk ID
- Description

- Probability
  - Impact
  - Prevention
  - Fallback
  - Owner
  - Trigger condition
- 

## # 9. Phase 3 Debugging Protocol

請建立 debug 規範。

至少包含：

### 9.1 如果 topology 跑不起來

怎麼檢查、如何降階。

### 9.2 如果 NewReno 跑不起來

怎麼檢查、如何降階。

### 9.3 如果 CUBIC 跑不起來

怎麼檢查、如何降階。

### 9.4 如果 BBR 跑不起來

怎麼檢查、如何降階。

### 9.5 如果 metrics 不合理

例如 throughput 為 0、delay 極端、loss 異常，如何處理。

### 9.6 如果 figures 無法產出

如何先用 summary table 支撐報告。

---

## # 10. Phase 3 Output Report Template

請產出一份 Phase 3 baseline report 的模板。

至少包含：

1. Objective
2. Toolchain Metadata

3. Topology Summary
4. Scenario Matrix
5. Baseline Methods
6. Metrics Definition
7. Results Summary
8. Figure Index
9. Limitations
10. Next Step to Phase 4

注意：

這是 report template，可以有「待填」欄位，不要假裝已有結果。

## # 11. 給 Antigravity 的 Phase 3 執行指令

請產出一段可以直接交給 Antigravity 的 Phase 3 指令。

這段指令必須：

- 明確要求先讀 OpenSpec change-01~04
- 明確要求只做 baseline benchmark
- 明確禁止 ns3-gym / DQN / PPO / IPFS / QUIC / multi-agent / multi-path
- 明確要求 NewReno / CUBIC 優先，BBR strongly recommended
- 明確要求 S1 / S2 優先
- 明確要求 raw logs / summary CSV / plots / metadata
- 明確要求遇到 BBR 或 RTT 問題時降階並記錄
- 明確要求完成後停止，等待 spec owner 驗收

## # 12. Phase 3 Review Checklist

請建立 Phase 3 驗收檢查表。

檢查表必須讓我能回答：

- 是否遵守 change-01~04？
- 是否只做 baseline？
- 是否完成 NewReno？
- 是否完成 CUBIC？
- BBR 是否完成或有 fallback note？
- S1 / S2 是否完成？

- raw logs 是否存在？
- summary CSV 是否存在？
- figures 是否存在？
- metadata 是否完整？
- README 是否更新？
- 是否沒有偷偷進入 ns3-gym？
- 是否沒有偷偷訓練 DQN？
- 是否能支撐 Phase 4？

---

## 六、輸出品質要求

### 1. 語言

- 全文使用繁體中文
- 必要英文專有名詞保留英文
- repo path、file name、artifact name 保持英文

### 2. 深度

不要只列大綱。

要寫到我可以直接拿去指揮 Antigravity 開工。

### 3. 嚴謹度

所有重要判斷都要有可信來源支撐。

若是推論，請明確寫「這是基於來源的推論」。

### 4. 禁止事項

你現在不能：

- 寫實際 ns-3 C++ code
- 寫 Python analysis code
- 寫 shell command
- 假裝已經跑出 baseline
- 假裝已有圖表
- 進入 Phase 4
- 設計 DQN training script

- 啟動 ns3-gym
- 擴張到 IPFS / QUIC / PPO / multi-agent / multi-path
- 宣稱已有實驗結果

## 七、現在開始執行

請先搜尋官方文件與正式來源，再輸出。

請直接從：

### Phase 3：Baseline 先行總結

開始。

以下是 Phase 3：Baseline 先行的「最完美且超高品質」最佳 prompt。

這一階段開始從 Phase 2 的 OpenSpec 文件進入可執行準備與 baseline 實作指揮，但核心仍不是直接衝 DQN，而是先讓 Antigravity 依照 change-02-baseline-benchmark 建立 ns-3 單一瓶頸鏈路 baseline。ns-3 官方 TCP model 支援 NewReno、CUBIC、BBR 等 congestion control 演算法；Flow Monitor 可提供 flow-level 統計，例如 tx/rx bytes、packets、lost packets、delaySum 等；ns-3 tracing 也被官方教學定位為從 simulation 取出資訊的重要機制。這些會被要求寫入 Phase 3 的執行規格與驗收條件。同時，OpenSpec 的目的就是讓人與 AI coding assistant 在寫 code 前先對齊規格，並以 change folder 集中管理 proposal、design、tasks、specs；所以 Phase 3 必須明確要求 Antigravity 先讀已驗收的 change-01~04，再只執行 baseline，不可跳到 ns3-gym / DQN / PPO。

請下一回合直接貼上以下整段：

你現在要執行的是我 DRL Final Project 的下一階段：

# Phase 3：Baseline 先行

## 最高任務目標

你要基於已驗收完成的 Phase 2 OpenSpec 四個 change：

- change-01-project-charter
- change-02-baseline-benchmark
- change-03-opengym-env
- change-04-drl-mvp

替我產出一份可以交給 **Antigravity** 執行的 **Phase 3：Baseline 先行實作指揮包**。

注意：這一階段不是訓練 DQN，不是寫 ns3-gym environment，不是進入 DRL。

這一階段的唯一目標是：

> 先把 ns-3 single bottleneck baseline benchmark 做出來，並產出可支撐後續 DQN comparison 的 logs、tables、figures 與 README 說明。

你要把 Phase 2 的 OpenSpec 規格轉成 Phase 3 的「執行指令、任務拆解、驗收條件、風險降階、Antigravity prompt」。

---

# 一、已完成前置階段

## 1. Phase 2 已驗收完成

以下 change 已通過：

### 01-project-charter

已凍結：

- 題目
- scope / non-goals
- toolchain
- baseline list
- metrics
- MVP definition
- downstream change map

### 02-baseline-benchmark

已凍結：

- NewReno / CUBIC / BBR baseline roles
- single bottleneck topology concept
- throughput / RTT or delay / loss / utility metrics
- scenario matrix
- output artifacts
- benchmark evaluation philosophy
- baseline-first 原則

### 03-opengym-env

已凍結：

- MDP interface
- environment boundary
- observation space
- discrete action space
- reward philosophy
- reset / step conceptual flow
- info dictionary
- random agent smoke test specification

### 04-drl-mvp

已凍結：

- DQN MVP boundary
- training protocol concept
- evaluation protocol concept
- baseline comparison protocol
- output artifacts
- success / failure criteria
- extension rules

Phase 3 必須完全承接上述內容，不得自行改題、擴題、跳到 DQN、跳到 PPO，或直接做 IPFS / QUIC / multi-agent / multi-path。

---

## ## 2. 正式題目

### ### 中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

### ### 英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

### ### GitHub repo name

drl-bottleneck-congestion-control

---

## # 二、Phase 3 的核心目的

Phase 3 的任務是建立真正可用的 **ns-3 baseline benchmark foundation**。

它要回答：

1. ns-3 baseline benchmark 應如何啟動？
2. single bottleneck topology 的 MVP 實作順序應如何安排？
3. NewReno / CUBIC / BBR 應如何分層執行？
4. 若 BBR 有風險，如何不阻塞 MVP？
5. throughput / RTT or delay / loss / utility 要如何被記錄與輸出？
6. raw logs、summary CSV、plots、metadata 要如何組織？
7. S1 / S2 / S3 / S4 scenario 應如何分層執行？
8. Phase 3 哪些成果可以直接支撐 Phase 4 DQN MVP？
9. 如果 baseline 結果異常，應如何 debug 與降階？
10. 要給 Antigravity 哪一段明確、可執行、不偏航的 implementation prompt？

---

## # 三、本階段允許與禁止事項

## ## 3.1 Phase 3 允許做什麼

Phase 3 允許規劃並交給 Antigravity 執行：

- 建立 ns-3 baseline benchmark 專案骨架
- 建立 single bottleneck topology
- 執行 NewReno baseline
- 執行 CUBIC baseline
- 嘗試執行 BBR baseline
- 若 BBR 有版本或設定風險，降階為 strongly recommended / optional
- 建立 baseline logging pipeline
- 匯出 raw logs
- 匯出 summary CSV
- 匯出 per-scenario summary table
- 產出 throughput plot
- 產出 RTT / delay plot
- 產出 packet loss plot
- 產出 utility comparison plot
- 建立 experiment metadata
- 更新 README 的 baseline 章節
- 產出 Phase 3 驗收報告

## ## 3.2 Phase 3 禁止做什麼

Phase 3 嚴格禁止：

- 不建立 ns3-gym environment
- 不寫 DQN training script
- 不啟動 DQN training
- 不接 PPO
- 不做 reward ablation
- 不做 observation ablation
- 不接 IPFS
- 不接 QUIC
- 不修改 Linux kernel TCP
- 不做 multi-agent RL
- 不做 multi-path routing
- 不做大型 topology
- 不把 Pantheon 設為必裝工具
- 不宣稱 DRL 結果
- 不把 baseline 結果硬解讀成 DRL 成果
- 不在未完成 baseline 前進入 Phase 4

---

## # 四、你必須先搜尋並引用的來源



請先做最新網路搜尋，不可只靠記憶。

## ## 優先使用來源

請優先搜尋並引用：

1. OpenSpec 官方網站 / GitHub README / docs
2. OpenSpec concepts / workflow 文件
3. ns-3 官方 TCP model 文件
4. ns-3 Flow Monitor 官方文件
5. ns-3 tracing / tutorial / statistics 官方文件
6. ns-3 官方 examples / tutorial，如有需要
7. ns3-gym 官方 GitHub / ns-3 App Store 頁面
8. Aurora 原始論文
9. Pantheon paper / benchmark 說明
10. 必要時補充正式論文或 RFC

## ## 你必須用來源確認的判斷

請確認並支撐以下判斷：

- ns-3 適合做 TCP / congestion control baseline
- ns-3 支援 NewReno、CUBIC、BBR 等 TCP congestion control
- Flow Monitor 可支援 flow-level statistics，適合 throughput / delay / loss 類量測
- ns-3 tracing 可用於更細緻的 simulation data extraction
- baseline-first 是後續 DQN comparison 的必要前置
- Pantheon 可作 benchmark / reproducibility 思想來源，但不是 Phase 3 必裝工具
- Phase 3 應輸出可重現、可追蹤、可視覺化的 baseline artifacts
- 如果 BBR 或 RTT logging 不穩，應採降階策略而非卡死整個專案

---

## # 五、這次必須輸出的內容格式

請嚴格按照以下章節輸出，不要跳章，不要偷省略。

---

## ## # Phase 3：Baseline 先行總結

請用 150-250 字說明：

- Phase 3 的目的
- 它如何承接 Phase 2 四個 OpenSpec changes
- 為什麼 baseline 必須先於 DQN
- Phase 3 的最小可行成果是什麼
- Phase 3 如何避免 Antigravity 直接跳到 DRL

---

## ## # 1. Phase 3 Mission / Boundary

請輸出：

### ### 1.1 Phase 3 Mission

請正式說明 Phase 3 的使命。

### ### 1.2 Phase 3 In Scope

列出可做事項。

### ### 1.3 Phase 3 Out of Scope

列出禁止事項。

### ### 1.4 Phase 3 Success Definition

請定義：

成功不等於什麼。

成功等於什麼。

---

## ## # 2. Phase 3 Repo / Directory Blueprint

請設計 Phase 3 建議 repo 結構。

至少包含：

```
text
drl-bottleneck-congestion-control/
 README.md
 openspec/
 src/
 ns3/
 scratch_or_examples/
 helpers/
 experiments/
 configs/
 raw_logs/
 summaries/
 metadata/
 figures/
 baseline/
 reports/
 phase3-baseline/
 scripts/
 analysis/
```

請對每個資料夾說明：

- purpose
- should contain
- should not contain
- acceptance criteria

注意：

這是 repo blueprint，不是實際建立檔案。

### # 3. Baseline Implementation Roadmap

請把 Phase 3 拆成清楚的 implementation roadmap。

至少包含以下階段：

#### 3.1 Step A: Environment / Toolchain Verification

目的：

- 確認 ns-3 版本
- 確認 TCP variants 可用
- 確認 Flow Monitor 或 tracing 可用
- 確認 repo 結構

輸出：

- toolchain metadata
- version notes
- risk notes

#### 3.2 Step B: Minimal Single Bottleneck Topology

目的：

- 建立 one sender / one receiver / one bottleneck link
- 建立 one long-lived TCP flow
- 不做 DRL、不做 ns3-gym

輸出：

- topology description
- topology metadata
- sanity check note

#### 3.3 Step C: NewReno Baseline

目的：

- 先跑最基礎 TCP baseline
- 驗證 logging pipeline

輸出：

- NewReno raw logs
- NewReno summary
- metrics sanity check

#### 3.4 Step D: CUBIC Baseline

目的：

- 跑主要 modern TCP baseline
- 和 NewReno 使用同一 metrics 口徑

輸出：

- CUBIC raw logs
- CUBIC summary
- NewReno vs CUBIC comparison table

#### 3.5 Step E: BBR Baseline

目的：

- 嘗試 BBR
- 若 BBR 有問題，不阻塞 MVP

輸出：

- BBR result or BBR fallback note
- version / availability note

### 3.6 Step F: Scenario Matrix Execution

目的：

- 先完成 S1 / S2
- 再視情況做 S3
- S4 僅 optional

輸出：

- per-scenario baseline summaries
- scenario metadata

### 3.7 Step G: Visualization and Reporting

目的：

- 產出可放入 PPT / README 的 baseline figures

輸出：

- throughput plot
- RTT / delay plot
- packet loss plot
- utility plot
- baseline summary table
- Phase 3 report

## # 4. Scenario Execution Plan

請根據 change-02 的 scenario matrix 設計 Phase 3 執行順序。

必須包含：

### 4.1 MVP-required Scenarios

- S1 Stable low-delay bottleneck
- S2 Stable high-delay bottleneck

### 4.2 Should-have Scenario

- S3 Variable bandwidth bottleneck

### 4.3 Optional Scenario

- S4 Bottleneck with cross traffic

請為每個 scenario 提供：

- priority
- purpose
- required baselines
- required metrics
- expected artifact
- risk
- fallback

## # 5. Metrics / Logging / Artifact Requirements

請定義 Phase 3 必須產出的 metrics 與 artifacts。

### 5.1 Metrics

至少包含：

- average throughput
- average RTT / delay

- packet loss rate
- utility score

## 5.2 Raw Logs

請定義 raw logs 應包含：

- scenario ID
- method
- run ID
- seed if applicable
- received bytes
- tx / rx packets
- delay / RTT related fields
- lost / dropped packets if available
- simulation duration
- metadata

## 5.3 Summary CSV

請定義 summary CSV 欄位。

至少包含：

- scenario\_id
- method
- run\_id
- throughput
- avg\_delay\_or\_rtt
- loss\_rate
- utility
- notes

## 5.4 Figures

至少包含：

- throughput comparison
- delay / RTT comparison
- loss comparison
- utility comparison

## 5.5 README / PPT-ready Summary

請定義後續 README / PPT 應如何使用 baseline 結果。

## # 6. Utility Score Specification

請定義 Phase 3 utility score 的執行口徑。

必須包含：

- 為什麼需要 utility
- 為什麼不能只看 throughput
- conceptual formula
- normalization requirement
- weights not final / can be revised
- 如何避免誤讀

請使用：

$$\text{utility} = \text{normalized\_throughput} - \text{penalty\_delay} - \text{penalty\_loss}$$

請提供：

- 符號注釋
- 白話說明
- 注意：若 Phase 3 不足以穩定定義 utility，可先輸出 raw metrics, utility 作 provisional / analysis metric

## # 7. Acceptance Criteria

請建立 Phase 3 驗收條件。

至少包含：

### 7.1 Minimum Acceptance

例如：

- repo structure ready
- S1 / S2 completed for NewReno and CUBIC
- raw logs available
- summary CSV available
- at least throughput and delay / RTT comparison available
- BBR fallback documented if unavailable

### 7.2 Strong Acceptance

例如：

- BBR completed
- S3 completed
- loss and utility plots available
- metadata complete
- README baseline section completed

### 7.3 Excellent Acceptance

例如：

- S4 completed
- repeat runs / seeds
- all figures PPT-ready
- limitation notes complete
- artifacts cleanly organized

## # 8. Risk Register and Fallback Strategy

請建立 Phase 3 專屬風險清單。

至少包含：

1. ns-3 安裝 / 版本問題
2. TCP variant 名稱或 API 不一致
3. BBR 不可用
4. Flow Monitor 無法取得 RTT
5. delay / RTT 口徑混淆
6. loss rate 計算不穩
7. throughput 計算時間窗不一致
8. scenario 太複雜
9. S3 / S4 失控
10. baseline 結果異常
11. 圖表無法支撐 PPT
12. Antigravity 直接跳到 DQN
13. repo artifacts 混亂
14. metadata 缺失

表格欄位請包含：

- Risk ID
- Description
- Probability
- Impact
- Prevention
- Fallback
- Owner
- Trigger condition

## # 9. Phase 3 Debugging Protocol

請建立 debug 規範。

至少包含：

9.1 如果 topology 跑不起來

怎麼檢查、如何降階。

9.2 如果 NewReno 跑不起來

怎麼檢查、如何降階。

9.3 如果 CUBIC 跑不起來

怎麼檢查、如何降階。

9.4 如果 BBR 跑不起來

怎麼檢查、如何降階。

9.5 如果 metrics 不合理

例如 throughput 為 0、delay 極端、loss 異常，如何處理。

9.6 如果 figures 無法產出

如何先用 summary table 支撐報告。

## # 10. Phase 3 Output Report Template

請產出一份 Phase 3 baseline report 的模板。

至少包含：

1. Objective
2. Toolchain Metadata
3. Topology Summary
4. Scenario Matrix
5. Baseline Methods
6. Metrics Definition
7. Results Summary
8. Figure Index
9. Limitations
10. Next Step to Phase 4

注意：

這是 report template，可以有「待填」欄位，不要假裝已有結果。

## # 11. 給 Antigravity 的 Phase 3 執行指令

請產出一段可以直接交給 Antigravity 的 Phase 3 指令。

這段指令必須：

- 明確要求先讀 OpenSpec change-01~04
- 明確要求只做 baseline benchmark
- 明確禁止 ns3-gym / DQN / PPO / IPFS / QUIC / multi-agent / multi-path
- 明確要求 NewReno / CUBIC 優先, BBR strongly recommended
- 明確要求 S1 / S2 優先
- 明確要求 raw logs / summary CSV / plots / metadata
- 明確要求遇到 BBR 或 RTT 問題時降階並記錄
- 明確要求完成後停止，等待 spec owner 驗收

## # 12. Phase 3 Review Checklist

請建立 Phase 3 驗收檢查表。

檢查表必須讓我能回答：

- 是否遵守 change-01~04？
- 是否只做 baseline？
- 是否完成 NewReno？
- 是否完成 CUBIC？
- BBR 是否完成或有 fallback note？
- S1 / S2 是否完成？
- raw logs 是否存在？
- summary CSV 是否存在？
- figures 是否存在？
- metadata 是否完整？
- README 是否更新？
- 是否沒有偷偷進入 ns3-gym？
- 是否沒有偷偷訓練 DQN？
- 是否能支撐 Phase 4？

## 六、輸出品質要求

### 1. 語言

- 全文使用繁體中文
- 必要英文專有名詞保留英文
- repo path、file name、artifact name 保持英文

### 2. 深度

不要只列大綱。

要寫到我可以直接拿去指揮 Antigravity 開工。

### 3. 嚴謹度

所有重要判斷都要有可信來源支撐。

若是推論，請明確寫「這是基於來源的推論」。

### 4. 禁止事項

你現在不能：

- 寫實際 ns-3 C++ code
- 寫 Python analysis code
- 寫 shell command
- 假裝已經跑出 baseline
- 假裝已有圖表
- 進入 Phase 4
- 設計 DQN training script
- 啟動 ns3-gym
- 擴張到 IPFS / QUIC / PPO / multi-agent / multi-path
- 宣稱已有實驗結果

## 七、現在開始執行

請先搜尋官方文件與正式來源，再輸出。

請直接從：



**Phase 3 : Baseline 先行總結**

開始。

顯示更多顯示較少

## Phase 3 : Baseline 先行總結

**Phase 3 : Baseline 先行** 的目的，是把 Phase 2 已驗收的 OpenSpec 規格轉成可交給 Antigravity 執行的 baseline 實作指揮包。這一階段只做 **ns-3 single bottleneck TCP baseline benchmark**，不碰 ns3-gym、不訓練 DQN、不導入 PPO。baseline 必須先於 DRL，因為後續 DQN 若沒有 NewReno / CUBIC / BBR 的比較基準，就無法判斷 throughput、RTT / delay、loss、utility 的 trade-off 是否有意義。ns-3 官方 TCP model 支援多種 TCP congestion control 演算法，包含 CUBIC、NewReno、BBR；Flow Monitor 可提供 flow-level 統計，包含 tx / rx bytes、packets、lost packets、delaySum 等；ns-3 tracing 也可用於從 simulation 取出更細緻事件資料。🌱 [NSNAM+2](#)

Phase 3 的最小可行成果是：完成 S1 / S2 場景下 NewReno 與 CUBIC baseline、產出 raw logs、summary CSV、至少 throughput 與 delay / RTT 比較、並留下 BBR fallback note。此階段會明確禁止 Antigravity 直接跳到 DQN / ns3-gym / PPO，確保 Phase 4 的 DRL MVP 是建立在可重現 baseline 之上。

## 1. Phase 3 Mission / Boundary

### 1.1 Phase 3 Mission

Phase 3 的使命是建立本專題第一個真正可執行的 **baseline benchmark foundation**。也就是說，先用 ns-3 建立 single bottleneck link，跑出傳統 TCP baseline 的基本資料，包含 NewReno、CUBIC，並在可行情況下加入 BBR。這些 baseline 會成為後續 DQN agent 的比較對象。

此階段不是為了證明哪個 TCP 最強，也不是為了開始 DRL，而是為了確認三件事：

1. single bottleneck topology 可以穩定運作；
2. throughput、delay / RTT、loss、utility 可以被一致量測；
3. 後續 Phase 4 的 DQN evaluation 有可信 reference point。

OpenSpec 的價值在於先讓人與 AI coding assistant 對齊規格，再進入實作；其 change folder 可集中管理 proposal、specs、design、tasks，適合本專題以階段方式推進。🔗 [GitHub+1](#)

### 1.2 Phase 3 In Scope

Phase 3 可做：

- 建立 ns-3 baseline benchmark 專案骨架
- 建立 single bottleneck topology

- 建立 one sender / one receiver / one bottleneck link
- 建立 one long-lived TCP flow
- 執行 NewReno baseline
- 執行 CUBIC baseline
- 嘗試 BBR baseline
- 建立 Flow Monitor / tracing logging pipeline
- 匯出 raw logs
- 匯出 summary CSV
- 匯出 per-scenario summary table
- 產出 throughput comparison plot
- 產出 delay / RTT comparison plot
- 產出 packet loss comparison plot
- 產出 provisional utility comparison plot
- 建立 experiment metadata
- 更新 README baseline section
- 產出 Phase 3 baseline report

## 1.3 Phase 3 Out of Scope

Phase 3 禁止：

- 不建立 ns3-gym environment
- 不寫 DQN training script
- 不啟動 DQN training
- 不接 PPO
- 不做 reward ablation
- 不做 observation ablation
- 不接 IPFS
- 不接 QUIC
- 不修改 Linux kernel TCP
- 不做 multi-agent RL
- 不做 multi-path routing
- 不做大型 topology
- 不把 Pantheon 設為必裝工具
- 不宣稱 DRL 結果
- 不把 baseline 結果硬解讀成 DRL 成果

- 不在 baseline 未完成前進入 Phase 4

Pantheon 可作為 benchmark / reproducibility 思想來源，因為它提供 congestion-control research 的 common benchmark algorithms、shared evaluation platform 與 public archive of results；但基於本專題完成率，Phase 3 不把 Pantheon full integration 當作必裝依賴。這是基於來源的推論。 [Pantheon+1](#)

## 1.4 Phase 3 Success Definition

成功不等於：

- 已經訓練 DQN
- 已經建立 ns3-gym
- 已經產生 DRL 結果
- 已經證明 CUBIC / BBR / NewReno 誰最好
- 已經完成所有 scenario
- 已經完成 Pantheon integration

成功等於：

- repo 結構能支撐 baseline artifacts
- S1 / S2 至少能跑 NewReno 與 CUBIC
- BBR 完成或有清楚 fallback note
- raw logs 存在
- summary CSV 存在
- throughput、delay / RTT、loss 指標口徑清楚
- 至少能產出 baseline comparison table
- 至少能產出 throughput 與 delay / RTT 比較圖
- metadata 可追蹤 scenario、method、run、seed、版本
- README 有 baseline section
- Phase 4 DQN MVP 可直接引用這些 baseline 結果

## 2. Phase 3 Repo / Directory Blueprint

建議 repo blueprint：

```
drl-bottleneck-congestion-control/
 README.md
 openspec/
 src/
 ns3/
 scratch_or_examples/
 helpers/
 experiments/
 configs/
 raw_logs/
```

```
summaries/
metadata/
figures/
baseline/
reports/
phase3-baseline/
scripts/
analysis/
```

Path	purpose	should contain	should not contain	acceptance criteria
README.md	專案入口與 baseline 說明	project overview、scope、baseline status、metrics、how to interpret results	不放 DQN 結果、不宣稱 DRL 成果	有 Phase 3 baseline section
openspec/	Phase 2 真相來源	change-01~04 文件	不放實驗 raw data	可追溯所有規格決策
src/ns3/scratch_or_examples/	ns-3 baseline 相關實作位置	single bottleneck baseline implementation artifacts	不放 DQN / ns3-gym / PPO	只包含 baseline 相關檔案
src/ns3/helpers/	ns-3 baseline 輔助邏輯位置	topology / metric helper 相關內容	不放 Python training script	helper 不引入 DRL
experiments/configs/	實驗設定	scenario config、baseline method config、run config	不放 raw logs	每個 run 可追蹤設定
experiments/raw_logs/	原始輸出	per-run raw logs、Flow Monitor XML 或等效 logs	不放整理後圖表	raw data 可追蹤 scenario / method / run
experiments/summaries/	整理後數據	summary CSV、per-scenario table、baseline comparison table	不放未處理巨大 raw dump	CSV 欄位一致
experiments/metadata/	可重現資訊	ns-3 version、TCP variant、run ID、seed、scenario metadata	不放圖片	metadata 足以支撐重現
figures/baseline/	baseline 圖表	throughput、delay / RTT、loss、utility plots	不放 reward curve	圖表只呈現 baseline
reports/phase3-baseline/	Phase 3 報告	baseline report、limitation note、figure index	不放 final report 假結果	報告可供 spec owner 驗收
scripts/analysis/	後處理分析腳本位置	summary / plotting scripts, 僅 Phase 3 baseline 用	不放 DQN training / PPO / ns3-gym	只處理 baseline logs

## 3. Baseline Implementation Roadmap

### 3.1 Step A : Environment / Toolchain Verification

## 目的：

- 確認 ns-3 版本
- 確認 TCP variants 可用
- 確認 NewReno / CUBIC / BBR 支援狀態
- 確認 Flow Monitor 可用
- 確認 tracing 可用
- 確認 repo 結構符合 Phase 3 blueprint

## 輸出：

- **toolchain metadata**
- **version notes**
- **tcp variant availability note**
- **Flow Monitor / tracing availability note**
- **risk notes**

## 驗收重點：

- 明確記錄 ns-3 版本
- 明確記錄 CUBIC 是否 default
- 明確記錄 NewReno / BBR 是否可用
- 若 BBR 有問題，先記錄，不阻塞 NewReno / CUBIC

ns-3 官方 TCP model 說明 native ns-3 TCP 支援完整雙向 TCP，並支援多種 congestion control 演算法，包含 CUBIC、NewReno、BBR 等；因此 Step A 的第一個任務是確認本機版本與官方支援對齊。  NSNAM

## 3.2 Step B : Minimal Single Bottleneck Topology

### 目的：

- 建立 one sender / one receiver / one bottleneck link
- 建立 one long-lived TCP flow
- 不做 DRL
- 不做 ns3-gym
- 不做 DQN

### 輸出：

- topology description
- topology metadata
- sanity check note

**驗收重點：**

- topology 必須是 single bottleneck
- 不得擴張到 large-scale topology
- 不得加入 multi-agent / multi-path
- 不得加入 IPFS / QUIC

### 3.3 Step C : NewReno Baseline

**目的：**

- 先跑最基礎 TCP baseline
- 驗證 logging pipeline
- 確認 throughput / delay / loss 能被產生或推導

**輸出：**

- NewReno raw logs
- NewReno summary CSV row
- NewReno metrics sanity check
- NewReno metadata

**驗收重點：**

- NewReno baseline 能完成 S1
- raw logs 可追蹤 method = NewReno
- summary 中至少包含 throughput 與 delay / RTT 類欄位
- 若 loss 無法穩定取得，要有 note

### 3.4 Step D : CUBIC Baseline

**目的：**

- 跑主要 modern TCP baseline
- 和 NewReno 使用同一 metrics 口徑
- 建立第一個 baseline comparison table

**輸出：**

- CUBIC raw logs
- CUBIC summary CSV row
- NewReno vs CUBIC comparison table
- CUBIC metadata

#### 驗收重點：

- CUBIC 與 NewReno 使用相同 scenario、time window、metrics schema
- 不得把 CUBIC 結果與 NewReno 用不同時間窗比較
- 若 CUBIC 是 ns-3 default，仍必須明確標示 method = CUBIC

## 3.5 Step E : BBR Baseline

#### 目的：

- 嘗試 BBR
- 讓 baseline 更有說服力
- 若 BBR 有版本或 API 風險，不阻塞 MVP

#### 輸出：

- BBR result 或 BBR fallback note
- BBR version / availability note
- 若成功，加入 summary CSV 與 comparison table
- 若失敗，記錄原因與降階

#### 驗收重點：

- BBR 是 strongly recommended，不是 hard blocker
- 若 BBR 不可用，MVP 仍可用 NewReno + CUBIC
- 不得因 BBR 卡住而停掉整個 Phase 3

## 3.6 Step F : Scenario Matrix Execution

#### 目的：

- 先完成 S1 / S2
- 再視情況做 S3
- S4 僅 optional

#### 輸出：

- per-scenario baseline summaries
- scenario metadata
- scenario limitation notes

#### 驗收重點：

- S1 / S2 是 Phase 3 MVP-required

- S3 是 should-have
- S4 是 optional
- S3 / S4 不得阻塞 Phase 3 最小可行成果

## 3.7 Step G : Visualization and Reporting

目的：

- 產出可放入 README / PPT / 10 分鐘影片的 baseline artifacts
- 為 Phase 4 DQN comparison 留下一致圖表口徑

輸出：

- throughput plot
- RTT / delay plot
- packet loss plot
- utility plot
- baseline summary table
- Phase 3 report

驗收重點：

- 圖表不得宣稱 DRL 結果
- 圖表只呈現 baseline
- 若圖表尚不足，至少要有 summary table 支撐報告

## 4. Scenario Execution Plan

### 4.1 MVP-required Scenarios

Scenario	Priority	Purpose	Required baselines	Required metrics	Expected artifact	Risk	Fallback
S1 Stable low-delay bottleneck	P0	最基本 sanity check, 確認 topology、TCP flow、logging 可用	NewReno、CUBIC ; BBR strongly recommended	throughput、delay / RTT、loss、utility	raw logs、summary CSV、throughput / delay comparison	Low	若 BBR 失敗，保留 NewReno + CUBIC
S2 Stable high-delay bottleneck	P0	觀察高延遲條件下 baseline trade-off	NewReno、CUBIC ; BBR strongly recommended	throughput、delay / RTT、loss、utility	per-scenario summary、	Medium	若 delay / RTT 難取得，先用 Flow



Scenario	Priority	Purpose	Required baselines	Required metrics	Expected artifact	Risk	Fallback
					comparison plot		Monitor delay 類統計

## 4.2 Should-have Scenario

Scenario	Priority	Purpose	Required baselines	Required metrics	Expected artifact	Risk	Fallback
S3 Variable bandwidth bottleneck	P1	測試 baseline 對變動容量的反應	NewReno、CUBIC ; BBR if available	throughput、delay / RTT、loss、utility	scenario summary、variable-bandwidth note	Medium	若設計過複雜，延後到 Phase 4 後或 future work

## 4.3 Optional Scenario

Scenario	Priority	Purpose	Required baselines	Required metrics	Expected artifact	Risk	Fallback
S4 Bottleneck with cross traffic	P2	測試干擾流量下 baseline 穩定性	NewReno、CUBIC ; BBR optional	throughput、delay / RTT、loss、utility	optional comparison table、limitation note	High	直接標為 optional，不阻塞 MVP

## 5. Metrics / Logging / Artifact Requirements

### 5.1 Metrics

Phase 3 至少要產出：

Metric	定義	用途
average throughput	接收資料量 / 量測時間	衡量傳輸效率
average RTT / delay	RTT 或 delay 類平均值	衡量延遲與 queueing effect
packet loss rate	lost / transmitted 或等效口徑	衡量壅塞與丟包
utility score	throughput、delay、loss 的綜合分析指標	避免只看 throughput

Flow Monitor 可提供 flow-level statistics，包含 tx / rx bytes、tx / rx packets、lost packets、delaySum 等，適合作為 throughput、delay、loss 量測基礎；若需要更細節事件，ns-3 tracing 透過 trace sources 與

trace sinks 取得 simulation 中的事件資料。  NSNAM+1

## 5.2 Raw Logs

raw logs 應包含：

- scenario ID
- method
- run ID
- seed if applicable
- received bytes
- transmitted bytes if available
- tx packets
- rx packets
- delay / RTT related fields
- lost / dropped packets if available
- simulation duration
- measurement start / end concept
- ns-3 version
- TCP variant
- topology metadata
- notes

## 5.3 Summary CSV

summary CSV 欄位至少包含：

欄位	說明
<b>scenario_id</b>	S1 / S2 / S3 / S4
<b>method</b>	NewReno / CUBIC / BBR
<b>run_id</b>	run identifier
<b>seed</b>	若有 repeat run
<b>throughput</b>	average throughput
<b>avg_delay_or_rtt</b>	delay 或 RTT 平均值
<b>loss_rate</b>	packet loss rate
<b>utility</b>	provisional utility
<b>duration</b>	simulation / measurement duration
<b>tcp_variant</b>	實際使用 TCP algorithm
<b>notes</b>	fallback / caveat

## 5.4 Figures

至少包含：

- throughput comparison
- delay / RTT comparison
- loss comparison
- utility comparison

建議圖表規格：

Figure	X-axis	Y-axis	Grouping
throughput comparison	scenario ID	throughput	method
delay / RTT comparison	scenario ID	delay / RTT	method
loss comparison	scenario ID	loss rate	method
utility comparison	scenario ID	utility	method

## 5.5 README / PPT-ready Summary

README / PPT 應使用 baseline 結果來說明：

- 本專題已完成 baseline-first foundation
- NewReno / CUBIC 是 MVP baseline
- BBR 若完成，可作強化比較
- 若 BBR 不可用，需明確寫 fallback
- 指標不只看 throughput，也看 delay / RTT、loss、utility
- baseline 結果將作為 Phase 4 DQN comparison 的 reference point

## 6. Utility Score Specification

### 6.1 為什麼需要 utility

utility 是為了把 throughput、delay、loss 放進同一個分析框架。只看 throughput 可能造成誤判：某 baseline throughput 高，但 delay / loss 也高，整體不一定比較好。

### 6.2 為什麼不能只看 throughput

壅塞控制本質是 trade-off 問題。高 throughput 若是用 queue buildup 或高 loss 換來，不能被單獨視為成功。Pantheon benchmark 的精神也是讓 congestion-control schemes 能在共同評估平台上比較，而不是只用單一片面指標評估。 [Pantheon+1](#)

## 6.3 Conceptual formula

$$\text{utility} = \text{normalized\_throughput} - \text{penalty\_delay} - \text{penalty\_loss}$$

## 6.4 符號注釋

- **normalized\_throughput**：正規化後吞吐量，越高越好。
- **penalty\_delay**：延遲懲罰，delay / RTT 越高，扣分越多。
- **penalty\_loss**：丟包懲罰，loss rate 越高，扣分越多。

## 6.5 白話說明

utility 的意思是：「傳得快很好，但如果是靠塞爆網路、延遲暴增或大量丟包換來，就要扣分。」

## 6.6 Normalization requirement

Phase 3 不應硬寫死最終權重。建議原則：

- throughput 需要依 scenario 內可比較尺度正規化
- delay / RTT 需要依同一 scenario 內尺度正規化
- loss rate 可用 0-1 ratio
- utility 先作 provisional / analysis metric
- 若 utility 不穩，先輸出 raw metrics，utility 標為 provisional

## 6.7 如何避免誤讀

- utility 不是唯一評分
- utility 只是輔助比較
- raw throughput / delay / loss 必須同時保留
- Phase 4 DQN evaluation 不得只看 utility 或 reward curve

## 7. Acceptance Criteria

### 7.1 Minimum Acceptance

### Phase 3 最低驗收：

- ☐ repo structure ready
- ☐ toolchain metadata recorded
- ☐ S1 completed for NewReno
- ☐ S1 completed for CUBIC
- ☐ S2 completed for NewReno
- ☐ S2 completed for CUBIC
- ☐ raw logs available
- ☐ summary CSV available
- ☐ throughput comparison available
- ☐ delay / RTT comparison available
- ☐ BBR completed or fallback documented
- ☐ no ns3-gym implementation
- ☐ no DQN training
- ☐ no PPO
- ☐ no IPFS / QUIC / multi-agent / multi-path

## 7.2 Strong Acceptance

### 強驗收：

- ☐ BBR completed for S1 / S2
- ☐ S3 completed
- ☐ loss metric available
- ☐ provisional utility available
- ☐ loss and utility plots available
- ☐ metadata complete
- ☐ README baseline section completed
- ☐ Phase 3 report completed

## 7.3 Excellent Acceptance

### 高品質驗收：

- ☐ S4 completed
- ☐ repeat runs / seeds available
- ☐ all figures PPT-ready

- ☐ limitation notes complete
- ☐ artifacts cleanly organized
- ☐ figure index completed
- ☐ baseline results can directly support Phase 4 DQN comparison

## 8. Risk Register and Fallback Strategy

Risk ID	Description	Probability	Impact	Prevention	Fallback	Owner	Trigger condition
P3-R1	ns-3 安裝 / 版本問題	Medium	High	先做 toolchain verification	固定可用版本, 記錄版本限制	Antigravity	ns-3 baseline example 無法執行
P3-R2	TCP variant 名稱或 API 不一致	Medium	High	Step A 先確認 TCP variant availability	只保留可用 variant, 記錄 fallback	Antigravity	指定 NewReno / CUBIC / BBR 失敗
P3-R3	BBR 不可用	Medium	Medium	BBR 設為 strongly recommended	NewReno + CUBIC 完成 MVP	Antigravity	BBR 無法啟用或結果不穩
P3-R4	Flow Monitor 無法取得 RTT	High	Medium	將 RTT / delay 分開記錄	用 Flow Monitor delay 類統計代替 RTT	Antigravity	無法取得 TCP RTT trace
P3-R5	delay / RTT 口徑混淆	Medium	High	summary CSV 欄位命名為 <b>avg_delay_or_rtt</b> 並加 notes	README 註明使用 delay proxy	Spec owner + Antigravity	不確定指標代表 RTT 或 delay
P3-R6	loss rate 計算不穩	Medium	Medium	保留 tx / rx / lost / dropped raw fields	先標記 loss 為 provisional	Antigravity	lost / dropped 欄位不一致
P3-R7	throughput 計算時間窗不一致	Medium	High	統一 measurement window 與 metadata	重跑該 scenario	Antigravity	同 scenario 不同 method 使用不同 duration
P3-R8	scenario 太複雜	Medium	Medium	S1 / S2 MVP-first	S3 / S4 延後	Spec owner	S3 / S4 佔用過多時間
P3-R9	S3 / S4 失控	High	Medium	S3 should-have、S4 optional	保留 S1 / S2 作 Phase 3 成果	Spec owner	variable / cross traffic 結果不可解釋
P3-R10	baseline 結果異常	Medium	Medium	每個 run 留 metadata	標記 limitation, 回查 topology / metrics	Antigravity	throughput 0、delay 極端、loss 異常

Risk ID	Description	Probability	Impact	Prevention	Fallback	Owner	Trigger condition
P3-R11	圖表無法支撐 PPT	Medium	Medium	先做 summary table, 再補 plot	用 table + limitation note 支撐報告	Spec owner	figure 缺失或不可讀
P3-R12	Antigravity 直接跳到 DQN	Medium	High	指令明確禁止 DRL	停止工作, 回到 Phase 3 review	Spec owner	出現 DQN / training / reward curve
P3-R13	repo artifacts 混亂	Medium	Medium	使用固定 directory blueprint	重新整理 artifacts index	Antigravity	raw logs、figures、metadata 無法對應
P3-R14	metadata 缺失	Medium	High	metadata 作為 minimum acceptance	不接受該 run, 補 metadata	Antigravity	無法追蹤 scenario / method / run

## 9. Phase 3 Debugging Protocol

### 9.1 如果 topology 跑不起來

檢查方向：

- 是否真的維持 one sender / one receiver / one bottleneck link
- 是否有 routing / addressing / application 設定缺口
- 是否 TCP flow 有啟動
- 是否 simulation duration 足夠
- 是否 Flow Monitor / tracing 在 topology 上正確掛載

降階策略：

- 先移除 S3 / S4
- 只保留 S1 minimal topology
- 只跑一種 TCP variant
- 先確認 packet 有 tx / rx, 再談 metrics

### 9.2 如果 NewReno 跑不起來

檢查方向：

- TCP variant 名稱是否符合 ns-3 版本

- 是否版本中支援 NewReno
- 是否 fallback 到 default TCP 但未記錄
- 是否 topology / application 本身有問題

降階策略：

- 先跑 default TCP 確認 topology
- 再回查 NewReno variant 設定
- 若 NewReno 長期不可用，記錄為 blocking risk，需 spec owner review

## 9.3 如果 CUBIC 跑不起來

檢查方向：

- ns-3 版本是否以 CUBIC 為 default
- 是否明確指定 TCP variant
- 是否和 NewReno 使用同一 topology / time window

降階策略：

- 先使用 default TCP 並記錄是否為 CUBIC
- 若 CUBIC 指定失敗但 default 為 CUBIC，可先以 default-CUBIC note 處理
- 若 CUBIC 不可用，Phase 3 不應繼續進入 DQN，需回報 spec owner

## 9.4 如果 BBR 跑不起來

檢查方向：

- ns-3 版本是否支援 BBR
- TCP variant 名稱是否正確
- BBR 是否需要額外設定
- BBR 是否在該 topology 下行為異常

降階策略：

- BBR 不阻塞 MVP
- 保留 NewReno + CUBIC
- 產出 BBR fallback note
- 在 README / Phase 3 report 標為 strongly recommended but unavailable / unstable

## 9.5 如果 metrics 不合理



常見異常：

- throughput 為 0
- delay 極端
- loss rate 超出合理範圍
- tx / rx packets 對不起來
- utility 和 raw metrics 矛盾

處理方式：

- 先看 raw logs，不先相信圖表
- 檢查 measurement window
- 檢查 flow start / end
- 檢查 summary CSV 計算口徑
- 檢查 scenario metadata
- 若 RTT 不可得，改用 delay proxy 並註明

## 9.6 如果 figures 無法產出

降階方式：

- 先用 summary CSV 支撐 Phase 3 report
- 先產出 baseline comparison table
- 圖表延後修
- README 先放 table 與 limitation note
- 不得因此跳去 DQN

## 10. Phase 3 Output Report Template

以下為 Phase 3 baseline report 模板，允許保留「待填」，不得假裝已有結果。

### Markdown

#### # Phase 3 Baseline Report

#### ## 1. Objective

本階段目標是建立 ns-3 single bottleneck TCP baseline benchmark，作為後續 DQN MVP comparison 的 reference point。

#### ## 2. Toolchain Metadata

- ns-3 version: 待填
- Flow Monitor availability: 待填
- tracing availability: 待填
- TCP variants checked: 待填
- NewReno availability: 待填

- CUBIC availability: 待填
- BBR availability: 待填
- notes: 待填

### ## 3. Topology Summary

- topology type: single bottleneck
- sender count: 1
- receiver count: 1
- bottleneck link: 待填
- traffic type: long-lived TCP flow
- queue / buffer setting: 待填
- notes: 待填

### ## 4. Scenario Matrix

Scenario ID	Description	Status	Notes
S1	Stable low-delay bottleneck	待填	待填
S2	Stable high-delay bottleneck	待填	待填
S3	Variable bandwidth bottleneck	待填	待填
S4	Bottleneck with cross traffic	待填	optional

### ## 5. Baseline Methods

Method	Role	Status	Notes
NewReno	required baseline	待填	待填
CUBIC	required baseline	待填	待填
BBR	strongly recommended baseline	待填	fallback if unavailable

### ## 6. Metrics Definition

- average throughput: 待填
- average RTT / delay: 待填
- packet loss rate: 待填
- utility score: provisional analysis metric

### ## 7. Results Summary

Scenario	Method	Throughput	Delay / RTT	Loss Rate	Utility	Notes
待填	待填	待填	待填	待填	待填	待填

### ## 8. Figure Index

Figure	File	Description	Status
Throughput comparison	待填	待填	待填
Delay / RTT comparison	待填	待填	待填
Loss comparison	待填	待填	待填
Utility comparison	待填	待填	待填

### ## 9. Limitations

- BBR limitation: 待填
- RTT / delay limitation: 待填
- loss measurement limitation: 待填
- scenario limitation: 待填

### ## 10. Next Step to Phase 4

Phase 4 may begin only after S1 / S2 baseline results for NewReno and CUBIC are available with raw logs, summary CSV, metadata, and at least basic comparison figures or tables.

## 11. 給 Antigravity 的 Phase 3 執行指令

You are working on Phase 3: Baseline First for the DRL final project.

**Project:**

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

**Repository:**

drl-bottleneck-congestion-control

Before doing any work, read and follow the approved OpenSpec artifacts:

- openspec/changes/01-project-charter
- openspec/changes/02-baseline-benchmark
- openspec/changes/03-opengym-env
- openspec/changes/04-drl-mvp

**Current goal:**

Implement only the ns-3 single bottleneck TCP baseline benchmark foundation.

**Allowed work:**

- Prepare the baseline benchmark repo structure.
- Verify ns-3 version and TCP variant availability.
- Implement a minimal single bottleneck topology.
- Run NewReno baseline first.
- Run CUBIC baseline second.
- Attempt BBR baseline if available.
- Prioritize S1 Stable low-delay bottleneck and S2 Stable high-delay bottleneck.
- Produce raw logs.
- Produce summary CSV.
- Produce experiment metadata.
- Produce baseline comparison tables.
- Produce throughput comparison plot.
- Produce delay / RTT comparison plot.
- Produce packet loss comparison plot if data is available.
- Produce provisional utility comparison plot if raw metrics are stable.
- Update README baseline section.
- Produce Phase 3 baseline report.

**Strictly forbidden:**

- Do not create ns3-gym environment.
- Do not write DQN training script.
- Do not train DQN.
- Do not introduce PP0.
- Do not run reward ablation.
- Do not run observation ablation.
- Do not implement IPFS.
- Do not implement QUIC.
- Do not modify Linux kernel TCP.
- Do not introduce multi-agent RL.
- Do not introduce multi-path routing.
- Do not build large-scale topology.
- Do not make Pantheon a required dependency.
- Do not claim any DRL result.

**Baseline priority:**

1. S1 + NewReno
2. S1 + CUBIC
3. S2 + NewReno
4. S2 + CUBIC
5. BBR for S1 / S2 if available
6. S3 only after S1 / S2 are stable
7. S4 optional only

**Fallback rules:**

- If BBR is unavailable or unstable, document the reason and continue with NewReno + CUBIC.
- If RTT is unavailable, use delay-related statistics and document the proxy.
- If loss metrics are unstable, keep raw tx/rx/lost/drop fields and mark loss as provisional.
- If figures fail, produce summary tables first.
- If any task starts drifting toward DQN or ns3-gym, stop immediately and return to spec owner review.

**Required outputs:**

- raw logs
- summary CSV

- metadata
- baseline comparison table
- throughput plot
- delay / RTT plot
- loss plot if available
- utility plot if stable
- README baseline section
- Phase 3 baseline report

After completing Phase 3 outputs, stop and wait for spec owner review.

## 12. Phase 3 Review Checklist

### Phase 3 驗收檢查表

- ☐ 是否遵守 `change-01-project-charter` ?
- ☐ 是否遵守 `change-02-baseline-benchmark` ?
- ☐ 是否遵守 `change-03-opengym-env`, 未提前實作 environment ?
- ☐ 是否遵守 `change-04-drl-mvp`, 未提前訓練 DQN ?
- ☐ 是否只做 baseline benchmark ?
- ☐ 是否完成 repo baseline 結構 ?
- ☐ 是否記錄 ns-3 version ?
- ☐ 是否記錄 TCP variant availability ?
- ☐ 是否完成 single bottleneck topology ?
- ☐ 是否完成 NewReno ?
- ☐ 是否完成 CUBIC ?
- ☐ BBR 是否完成或有 fallback note ?
- ☐ S1 是否完成 ?
- ☐ S2 是否完成 ?
- ☐ S3 是否只在 S1 / S2 穩定後才執行 ?
- ☐ S4 是否未阻塞 MVP ?
- ☐ raw logs 是否存在 ?
- ☐ summary CSV 是否存在 ?
- ☐ metadata 是否完整 ?
- ☐ throughput comparison 是否存在 ?
- ☐ delay / RTT comparison 是否存在 ?
- ☐ loss comparison 是否存在或有 limitation note ?
- ☐ utility comparison 是否存在或標為 provisional ?
- ☐ README baseline section 是否更新 ?
- ☐ Phase 3 baseline report 是否完成 ?
- ☐ 是否沒有偷偷進入 ns3-gym ?

- ☐ 是否沒有偷偷訓練 DQN ?
- ☐ 是否沒有導入 PPO ?
- ☐ 是否沒有加入 IPFS ?
- ☐ 是否沒有加入 QUIC ?
- ☐ 是否沒有加入 multi-agent ?
- ☐ 是否沒有加入 multi-path ?
- ☐ 是否能支撐 Phase 4 DQN MVP comparison ?

## Phase 3 結論

**Phase 3 的唯一主線是 baseline-first。**

最小可行成果不是「漂亮 DRL」，而是先把 NewReno / CUBIC 在 S1 / S2 的 ns-3 baseline 資料做出來，留下可追蹤 logs、summary CSV、metadata 與初步圖表。BBR 很有價值，但不能阻塞 MVP。S3 / S4 可以加分，但不能拖垮主線。Phase 3 驗收通過後，才適合進入 **Phase 4 : DRL MVP Implementation**。

<https://github.com/donuop35/ns3-drl-congestion-control>

這 github 是我期末專題提案報告的目前成果

我想先確定目前實作方向沒有偏掉 還穩穩在可以跟 Antigravity 完美協作的方向

我想確認的是我現在這個 github 專案你應該可以看得出來我是不是在用官方的 OpenSpec 實現 SDD 規格開發

因為我之前發生過我跟一位 AI Agent 協作 我也是請他用官方 OpenSpec 套件用 SDD 開發模式幫我實作專案

結果他竟然用的不是官方 OpenSpec 我問他為什麼並且一直以來他用的是什麼

他竟然跟我說他是去模擬 OpenSpec 形容的 SDD 開發模式 他自己再用一樣的方式刻出來

等於只是複製了 OpenSpec SDD 其中的 workflow 而已

根本沒有真正用到 OpenSpec 套件工具

他跟我說這叫什麼棕色模擬 我不管他叫什麼色

他就是學了皮膚再自己的方式去套用 這方法一點都沒有經過實戰驗證 至少 OpenSpec 是有大量使用社群

自己亂規範自己 寫出來的程式一點可信度都沒有

我要你幫我檢查我這份 github 是否陷在一樣的泥沼裡

因為我要使用的 OpenSpec 套件 我也會叫 Antigravity 幫我安裝

我跟 Antigravity 協作的方式要以官方 OpenSpec 套件當作 SDD 的實現工具

OpenSpec 也是我跟 Antigravity 對齊專案方向的重要規格

請幫我仔細判斷

我要用的是「\*\*官方 OpenSpec 套件\*\*」

顯示更多顯示較少

我仔細看過你目前這份 GitHub。結論很明確：

**你目前這個 repo 還沒有真正進入「官方 OpenSpec 套件初始化後的 SDD 狀態」。**

但它也沒有假裝自己已經是正式 OpenSpec 專案，目前比較準確的定位是：

**Proposal / PDR 階段的 OpenSpec Preview，不是官方 OpenSpec initialized project。**

所以它現在不算陷入你說的那種「假裝用了 OpenSpec、實際只是 AI 自己模擬 workflow」的泥沼；但如果下一步 Antigravity 沒有真的安裝官方 OpenSpec、沒有執行 `openspec init`、沒有用 `/opsx:* workflow`，那就會開始掉進同樣風險。

## 1. 我目前看到的 repo 狀態

你的 GitHub root 目前主要有：

- `assets/`
- `docs/`
- `openspec-preview/`
- `pdr/`
- `proposal/`
- `.gitignore`
- `README.md`

GitHub 頁面顯示的是 `openspec-preview`，不是正式的 `openspec/` 目錄；README 也明確寫現在是「提案階段 / OpenSpec 預覽 / 尚未開始程式實作」。 [GitHub+1](#)

這點很重要，因為你目前 repo 自己有講清楚：

`openspec-preview/` 只是 AI 執行路線圖草案，不是正式 OpenSpec 初始化後的規格目錄。README 也寫「正式的 OpenSpec 初始化、ns-3 環境建置與 DQN 模型實作將於 Phase 2 展開」。 [GitHub](#)

## 2. 是否已經真的使用官方 OpenSpec 套件？

目前看起來：尚未。

官方 OpenSpec 的 Quick Start 要求是：

```
npm install -g @fission-ai/openspec@latest
openspec init
```

接著使用 `/opsx:propose <what-you-want-to-build>` 之類的 OpenSpec slash command 開始建立 change。官方 README 也示範 OpenSpec 會建立類似 `openspec/changes/add-dark-mode/` 的 change folder，裡面包含 `proposal.md`、`specs/`、`design.md`、`tasks.md`。 [GitHub+1](#)

但你的 repo 現在看到的是：

檢查項目	目前 repo 狀態	判斷
是否有正式 <code>openspec/</code> 目錄	目前只看到 <code>openspec-preview/</code>	尚未正式初始化
是否有 <code>openspec/changes/...</code>	未看到	尚未進入官方 change workflow
是否有 <code>proposal.md</code> / <code>design.md</code> / <code>tasks.md</code> / <code>specs/</code> 的正式 change 結構	未看到正式 OpenSpec 目錄下的版本	目前只是 preview
是否有官方 OpenSpec CLI 初始化痕跡	GitHub 上看不到	無法確認已安裝 / 已初始化
README 是否誠實標示狀態	有，寫明是 OpenSpec 預覽、尚未正式實作	方向是誠實的

所以目前不是「用了官方 OpenSpec」，而是「準備要轉進官方 OpenSpec」。

### 3. 目前有沒有偏掉？

研究方向沒有偏掉。

README 的題目、MDP、state/action/reward、single bottleneck、排除 IPFS / QUIC / multi-agent 等內容，都和我們前面凍結的方向一致。README 寫到本專案目標是將 congestion control 形式化為 DRL 序列決策問題，並且 state、action、reward 也維持在我們定義的架構內。 [GitHub](#)

但 **OpenSpec 實作方式還沒到位**。

你現在有的是「OpenSpec preview / roadmap」，不是「官方 OpenSpec 套件產出的正式 SDD 專案骨架」。

我會把目前狀態評為：

方向正確，治理意識正確，但官方 OpenSpec 尚未落地。

### 4. 你最該警惕的點

你前面擔心的情況是：

AI Agent 只模仿 OpenSpec 的 workflow，自己刻一套 Markdown 規格流程，但沒有真正使用官方 OpenSpec 套件。

以目前 repo 來看，它還沒有到那一步，因為它明確叫做 **openspec-preview**，而且 roadmap 也寫明這不是正式 **openspec/** directory，正式 OpenSpec initialization 之後才開始。 

真正危險會發生在下一步：

如果 Antigravity 接下來只是手動建立：

```
openspec/
 changes/
 01-project-charter/
 proposal.md
 design.md
 tasks.md
 specs/
```

但沒有先安裝官方 OpenSpec、沒有執行 `openspec init`、沒有使用 `/opsx:propose` 或相關官方 workflow，那就會變成你說的「學皮膚，但沒有用官方套件」。

## 5. 下一步我建議你強制要求 Antigravity 做的事

你要對 Antigravity 下這段硬性要求：

你必須使用官方 OpenSpec 套件，不得自行模擬 OpenSpec workflow。

請先執行並回報以下事項：

1. 確認 Node.js 版本符合 OpenSpec 官方需求。
2. 安裝官方套件：  
`npm install -g @fission-ai/openspec@latest`
3. 在專案根目錄執行：  
`openspec init`
4. 回報 `openspec init` 後實際產生的檔案與資料夾。
5. 回報 OpenSpec slash command 是否可用，例如 `/opsx:propose`。
6. 不得手動偽造 `openspec/changes` 目錄來冒充 OpenSpec 初始化結果。
7. 不得只依照 README 或 `openspec-preview` 自行刻出一套 OpenSpec-like workflow。
8. 若 official OpenSpec CLI 無法安裝或無法執行，必須停止，不得繼續實作。

官方 OpenSpec README 明確寫安裝方式是 `npm install -g @fission-ai/openspec@latest`，初始化方式是 `openspec init`，後續才是 `/opsx:propose`；所以你可以把這三個動作設成「不得跳過」的驗收條件。 

## 6. 我建議你現在 repo 要補的一段 README 聲明

你可以在 README 的 OpenSpec 區塊補上這段，避免老師、Antigravity 或未來的你誤判：

### Markdown

#### ## OpenSpec Status

This repository currently contains only an OpenSpec Preview for proposal-stage planning.

The official OpenSpec workflow has not yet been initialized in this repository.

Before any implementation begins, Antigravity must install and use the official OpenSpec package:

```
- `npm install -g @fission-ai/openspec@latest`
- `openspec init`
```



```
- ` /opsx:propose ...`
```

The `openspec-preview/` directory must not be treated as a substitute for the official `openspec/` directory generated by OpenSpec.

這段很關鍵，因為它會直接堵住「AI 自己模擬 OpenSpec」的灰色空間。

## 7. 最終判斷

目前沒有偏題。

你的 DRL final project 題目、研究邊界、baseline-first、DQN MVP、排除 IPFS / QUIC / multi-agent 的方向都還是穩的。

但目前也還沒有正式使用官方 OpenSpec 套件。

現在只是 proposal 階段的 openspec-preview，而且 repo 自己有誠實標示這點。這是可以接受的。

下一步一定要讓 Antigravity 做官方 OpenSpec 初始化。

只要 Antigravity 沒有真的執行 `npm install -g @fission-ai/openspec@latest`、`openspec init`、並使用 `/opsx:* workflow`，就不能算正式 OpenSpec SDD。官方 OpenSpec 的核心就是先用 specs 對齊再寫 code，並以每個 change 的資料夾管理 proposal、specs、design、tasks；這不是單純 Markdown 排版可以取代的。  [GitHub](#)

你之前幫我生成的四階段 OpenSpec 規範

你是以 OpenSpec 這個官方套件下去考慮而生成的內容嗎

還是你也是自己去模擬 OpenSpec 的 SDD workflow

隨便你爽寫一些規則下來呢

顯示更多顯示較少

Phase 5：做 DQN MVP

截圖是 Phase 4：DRL MVP Implementation 的 commit

Antigravity 做完「Phase 4：DRL MVP Implementation」 **最佳 Prompt** 指令了

並已推上 github Phase 4：DRL MVP Implementation 的異動 請幫我驗收 git diff

是否全部 **Phase 4：DRL MVP Implementation** 要求都有通過

是否能推進到「**Phase 5：做 DQN MVP**」了呢

顯示更多顯示較少

## 驗收結論

我重新用 GitHub connector 直接抓了你給的兩個 commit，這次可以確認：

```
2e3b253aa6472e42af536f260154397026745777
→ 465959e31ed8fc9b3a4ae774fc080f624d1d9579
```

這段 diff 確實存在，而且 465959e 是在 2e3b253 之後的 1 個 commit。commit message 顯示 Antigravity 補上了 **S2 DQN training**、**S2 evaluation**、**seed sensitivity**、**CSV 更新**、**S1/S2 comparison figures**、**S2 episode logs**、**Excellent Acceptance report** 與 **README 更新**。

[Phase 4 Excellent Acceptance CO...](#)

但我的最終判定是：

```
Phase 4 Excellent Acceptance: PARTIAL PASS
Phase 5 Final Reporting / Demo / PPT Package: 暫時 NO-GO
```

原因很明確：**核心實驗成果已經補齊**，但治理文件與部分 **artifact index / smoke report** 還沒有同步乾淨。如果你堅持「Excellent Acceptance」才進 Phase 5，那現在還差一個小修正 commit。

## 1. 已通過的部分：核心 DQN Excellent 成果已補齊

### S2 DQN training / evaluation 已完成

`phase4-excellent-acceptance-report.md` 明確寫 S2 DQN 已完成 30k steps、訓練約 18.9 分鐘，`ep_rew_mean` 從 55.0 提升到 86.5，並做了 5 episodes deterministic evaluation。

[phase4-excellent-acceptance-rep...](#)

`dqn_summary.csv` 也已經不是 S1-only，現在有 S1 與 S2 兩列 DQN evaluation 結果。S2 結果為 throughput 9.786 Mbps、avg delay 148.816 ms、loss 5.537%、utility 0.7574、5 episodes deterministic。

[dqn\\_summary](#)

### DQN vs baseline comparison 已補上 S1 / S2

`dqn_vs_baseline_summary.csv` 已包含 S1 / S2 的 NewReno、CUBIC、BBR，以及 S1 / S2 的 DQN，共 12 列。S2 DQN 已放進比較表，utility 0.7574，低於 NewReno 0.9227 與 CUBIC 0.8182，但高於 BBR S2 anomaly 的 -0.1692。

[dqn\\_vs\\_baseline\\_summary](#)

這一點很重要：Antigravity 沒有硬包裝 DQN 全勝，Excellent report 也誠實寫出 S2 DQN ranks 3rd，主因是高 RTT 場景下 loss rate 達 5.54%。

[phase4-excellent-acceptance-rep...](#)

## action distribution 與 seed sensitivity 已補上

`dqn_action_distribution_summary.csv` 已存在 S1 / S2 兩列。S1 是 100% increase ; S2 則是 13.13% decrease、0% keep、86.87% increase。

[dqn\\_action\\_distribution\\_summary](#)

`dqn_seed_sensitivity_summary.csv` 也已包含 S1 / S2，各自 eval seeds 42 / 43 / 44；它清楚標註這是 **eval-only seed sensitivity; not a full retraining study**，這符合我們要求的「mini-check 不可誇大」。

[dqn\\_seed\\_sensitivity\\_summary](#)

## Excellent report 本身有列出 PASS

Excellent report 的 gap table 宣稱 GAP-01 到 GAP-10 全部完成，包含 S2 DQN、README、OpenSpec tasks、real-ZMQ、seed sensitivity、artifact-index、S2 action distribution、S2 comparison figures 等。

[phase4-excellent-acceptance-rep...](#)

Acceptance Criteria 也宣稱 S1/S2 training、S1/S2 eval、real-ZMQ、S1/S2 comparison figures、seed sensitivity、README、OpenSpec tasks、artifact-index、honest reporting 全部完成。

[phase4-excellent-acceptance-rep...](#)

## 2. 沒通過的地方：文件同步仍有明顯矛盾

這就是我不放行 Phase 5 的原因。

### A. OpenSpec tasks.md 仍顯示 S2 pending

`openspec/changes/dqn-mvp-agent/tasks.md` 的 Section 14 還是舊狀態。它明確寫：

- S2 DQN PENDING
- S2 eval PENDING
- S2 comparison pending
- S2 reward curve / action distribution / comparison figures pending
- artifact-index pending
- phase4-excellent-acceptance-report pending

這和 Excellent report 宣稱「全部完成」互相矛盾。

[tasks](#)

這是 **Excellent Acceptance blocker**。

因為你的專案強調 OpenSpec 是 source of truth，不能讓 report 說 PASS，但 OpenSpec tasks 還停在 pending。

## B. artifact-index.md 仍顯示大量 pending

**artifact-index.md** 也還是舊狀態。它把 S2 model、S2 Monitor CSV、S2 eval CSV、DQN summary S2 row、seed sensitivity、action distribution、S2 reward curve、S2 action distribution、S2 comparison figures 全部標成 pending / in progress。

artifact-index

artifact-index artifact-index

更明顯的是，artifact-index 的 Key Result Summary 還把 S2 DQN 顯示為 ，並在最後寫「Phase 4 Excellent Acceptance Upgrade in progress」。

artifact-index

這也是 **Excellent Acceptance blocker**。

你要求 artifact index 是 Phase 5 的素材地圖，現在這份 index 不能直接給 Phase 5 用。

## C. smoke-test-report.md 沒有更新成 hardened real-ZMQ 版

程式碼本身已經有 real-ZMQ hardening：`allow_dummy=False` 預設、ns3gym 不可 import 時直接 RuntimeError、ZMQ mode 非 real 時 fail、throughput non-zero check 等。

ns3\_congestion\_env

smoke\_test smoke\_test

但 committed 的 **smoke-test-report.md** 還是舊報告。它沒有記錄 `HAS_NS3GYM=True`、沒有記錄 `ZMQ mode=real`，而且 limitations 仍寫「如果 ns3gym 沒裝，會跑 dummy observation」。

smoke-test-report

smoke-test-report

這代表：**code 已修，但正式報告沒有重新生成或沒有 commit 新版**。

以 Excellent Acceptance 標準來看，仍未完成。

## D. Excellent report 內部也有一處未同步

Excellent report 的 S2 action distribution 區塊仍寫「Pending confirmation from `dqn_action_distribution_summary.csv`」，但實際 CSV 已經有 S2 action distribution。

phase4-excellent-acceptance-rep...

dqn\_action\_distribution\_summary

這是小問題，但對 final report / PPT 會造成困擾，應該修掉。

## E. dqn\_summary.csv 有一個 reproducibility 小問題

S1 的 `model_path` 是本機絕對路徑：

```
/mnt/c/Users/donuop/Documents/grassland/...
```

S2 則是 repo-relative path。

[dqn\\_summary](#)

這不影響實驗結果本身，但會影響 repo 可攜性與 Phase 5 整理。建議把 S1 也改成：

```
experiments/drl/models/dqn_s1_seed42.zip
```

### 3. 驗收表

要求	結果	判定
S1 DQN training / eval	已完成	✓
S2 DQN training / eval	已完成	✓
S1 / S2 DQN vs baseline	已完成，CSV 有 S1/S2 DQN	✓
S1 / S2 action distribution	CSV 有 S1/S2	✓
seed sensitivity mini-check	已完成，且標註 eval-only	✓
honest reporting	S1 不贏 BBR、S2 輸 NewReno/CUBIC 有誠實寫出	✓
不偷渡 PPO / IPFS / QUIC / multi-agent	commit message 與 report 都維持禁止	✓
real-ZMQ code hardening	code 已完成	✓
smoke-test-report 實際同步	仍是舊版，未記錄 HAS_NS3GYM / zmq_mode	✗
OpenSpec tasks.md 同步	仍顯示 S2 pending	✗
artifact-index.md 同步	仍顯示 S2 / figures / report pending	✗
Excellent report 內部一致性	S2 action distribution 仍寫 pending confirmation	⚠
repo-relative artifact path	S1 model path 仍是本機絕對路徑	⚠

### 4. 最終判定

Phase 4 Excellent Acceptance Upgrade + Current Fixes: PARTIAL PASS  
Phase 5 Final Reporting / Demo / PPT Package: NO-GO for now

不是因為 DQN 沒完成。

**DQN S2 補強已經完成，核心實驗成果是 OK 的。**

真正卡住的是：

**Excellent Acceptance 的治理與文件一致性還沒乾淨。**

你這個專案很重視 OpenSpec / SDD / artifact traceability，所以不能讓 `phase4-excellent-acceptance-report.md` 說全部 PASS，但 `tasks.md`、`artifact-index.md`、`smoke-test-report.md` 還在舊狀態。

## 5. 請 Antigravity 補一個 final cleanup commit

我建議你要求 Antigravity 補一個很小但關鍵的 commit：

**phase4: final excellent acceptance cleanup and artifact sync**

只要做以下 5 件事：

1. **更新** `openspec/changes/dqn-mvp-agent/tasks.md`  
把 S2 training/eval/comparison/figures/artifact-index/excellent report 全部改成完成；保留 `/opsx:apply deferred` 不動。
2. **更新** `reports/phase4-drl-mvp/artifact-index.md`  
把所有 S2 pending 改成完成，Key Result Summary 加入 S2 DQN 9.786 Mbps / 148.8 ms / 5.54% / utility 0.757，最後狀態改成 **Phase 4 Excellent Acceptance Complete**。
3. **重新生成並 commit** `reports/phase4-drl-mvp/smoke-test-report.md`  
必須包含：
  - `HAS_NS3GYM=True`
  - `S1 ZMQ mode=real`
  - `S2 ZMQ mode=real`
  - `real-ZMQ enforcement enabled`
  - `throughput-nonzero check passed`
  - 不得再寫「dummy observation if ns3gym not installed」這種舊 limitation。
4. **修正** `phase4-excellent-acceptance-report.md` 的 **S2 action distribution 區塊**  
用實際 CSV 值：S2 decrease 13.13%、keep 0%、increase 86.87%，不要再寫 pending confirmation。
5. 把 `dqn_summary.csv` 的 **S1 model\_path** 改成 **repo-relative path**  
從 `/mnt/c/Users/.../dqn_s1_seed42.zip` 改成 `experiments/drl/models/dqn_s1_seed42.zip`。

## 6. 補完後能不能進 Phase 5？

補完上述 cleanup 後，我預期可以判定：

**Phase 4 Excellent Acceptance: PASS**  
**Phase 5 Final Reporting / Demo / PPT Package: GO**

目前距離 GO 很近，已經不是技術實作問題，而是**最後一層規格治理與文件同步問題**。

好 幫我修正好 Phase 4 規格治理與文件同步問題

給我可以讓 Antigravity 最精準執行修正任務的「最完美且超高品質的\*\*最佳 Prompt\*\*」

可以。這次修正任務的目標很明確：**不是重跑 DQN，也不是進 Phase 5，而是把 Phase 4 的規格治理與文件同步補乾淨**。我前次驗收的 blocker 是：`tasks.md`、`artifact-index.md`、`smoke-test-report.md` 與 `phase4-excellent-acceptance-report.md` 仍存在狀態不一致；但核心 S2 DQN、comparison、seed sensitivity 與 action distribution 已經存在。`dqn_summary.csv` 已有 S1/S2 DQN 結果，`dqn_vs_baseline_summary.csv` 也已有 S1/S2 baseline + DQN 比較列。

[dqn\\_summary](#)

[dqn\\_vs\\_baseline\\_summary](#)

另外，OpenSpec 官方強調每個 change 應以 proposal、specs、design、tasks 組織，讓人與 AI coding assistant 在寫 code 前先對齊；所以這次修正要把 **OpenSpec tasks 與實際 artifacts 狀態同步**，不能讓 report 說 PASS、tasks 卻還 pending。 

以下可以直接貼給 Antigravity。

<https://github.com/donuop35/ns3-drl-congestion-control/commit/900f4cbfff0864e702051d3371f7367ead99b05c>

Antigravity 做完**最佳 Prompt** 指令了

並已推上 github Phase 4 的異動 請幫我驗收 git diff

是否全部 **Phase 4** 要求都有通過 是否能推進到「**Phase 5：做 DQN MVP**」了呢

並幫我評估在做 Phase 5 之前

是否要先做 OpenSpec 的規格定義 Change 05: reporting-figures-and-demo  
顯示更多顯示較少

## 驗收結論

我這次重新驗收 commit：

**900f4cbfff0864e702051d3371f7367ead99b05c**

結論是：

**Phase 4 Excellent Acceptance: PASS**  
**Phase 5 Final Reporting / Demo / PPT Package: G0**

但我要先校正一個命名：**Phase 5 不應該再叫「做 DQN MVP」**。  
因為 DQN MVP 已經在 Phase 4 完成，而且已經補到 Excellent Acceptance。Phase 5 應正式命名為：

**Phase 5: Final Reporting / Demo / PPT Package**

也就是把 Phase 4 的 DQN MVP 成果整理成期末報告、簡報圖表、demo script、10 分鐘影片素材與 final README，而不是再開一輪 DQN 實作。

## 1. 這次 commit 解決了我前次卡住的 blocker

這次 commit message 是：

**phase4: final excellent acceptance cleanup and artifact sync**

diff 明確修正了我前次指出的核心問題：**dqn\_summary.csv** 的 S1 model path 從本機絕對路徑改為 repo-relative path；**tasks.md** 從 S2 pending 改成 S1/S2 DQN training、evaluation、comparison、figures、reports 全部完成；artifact index 也從 in progress 改為 Phase 4 Excellent Acceptance Complete。

[phase4: final excellent accepta...](#)

這代表這次不是單純補文字，而是把 **OpenSpec governance**、**artifact traceability**、**README / report / CSV 狀態一致性** 做了同步。

## 2. Phase 4 要求驗收表

驗收項目	結果	判定
S1 DQN training / evaluation	<b>tasks.md</b> 顯示 S1 30k steps、eval 5 episodes 完成	✓
S2 DQN training / evaluation	<b>tasks.md</b> 顯示 S2 30k steps、eval 5 episodes 完成	✓
S1 / S2 DQN summary	<b>dqn_summary.csv</b> 有 S1 / S2 兩列，且 model path 都是 repo-relative	✓
S1 / S2 baseline comparison	artifact index 顯示 <b>dqn_vs_baseline_summary.csv</b> 12 rows, S1/S2 × 4 algos	✓
S1 / S2 figures	artifact index 顯示 S1/S2 comparison figures、reward curves、action distributions 都完成	✓
seed sensitivity mini-check	artifact index 顯示 6 rows, std=0.000, 並標明 eval-only	✓
smoke test real-ZMQ hardening	smoke report 顯示 <b>HAS_NS3GYM=True</b> 、 <b>ZMQ mode=real</b> 、 <b>allow_dummy=False</b> default、throughput non-zero check	✓
OpenSpec tasks status	Section 14 已同步 S1/S2 training/eval/comparison/figures/reports 完成	✓
artifact-index 狀態	已改成 Phase 4 Excellent Acceptance Complete	✓
Honest reporting	report 明確寫 DQN S1 輸 BBR、S2 輸 NewReno/CUBIC，沒有過度宣稱	✓
Scope enforcement	<b>tasks.md</b> 明確保留 no PPO / no IPFS / no QUIC / no multi-agent / no multi-path / no fake data	✓
/opsx:apply	仍保留 deferred，沒有亂 archive	✓

## 3. 這次最關鍵的修正已完成



## A. OpenSpec tasks.md 已同步

**tasks.md** 現在明確列出：

- S1 / S2 DQN training complete ；
- S1 / S2 deterministic evaluation complete ；
- S1 / S2 baseline comparison complete ；
- S1 / S2 reward curve、action distribution、comparison figures complete ；
- smoke-test-report、artifact-index、phase4-excellent-acceptance-report complete ；
- no PPO / IPFS / QUIC / multi-agent / multi-path / fake data ；
- `/opsx:apply deferred` 保留。

[tasks](#)

這點很重要，因為你的專案核心是 OpenSpec / SDD，tasks.md 不能再和實作狀態不一致。這次已經修正。

## B. artifact-index.md 已可支撐 Phase 5

**artifact-index.md** 現在列出 S1 / S2 model、training logs、evaluation CSV、summary CSV、seed sensitivity、action distribution、comparison figures、reports 與 source code，且全部標為 complete。它也清楚列出 key result summary：S1 DQN utility 0.900、S2 DQN utility 0.757，並誠實註明 S2 high loss 5.54% 代表 DQN 在 high-RTT 條件下仍偏向追 throughput。

[artifact-index](#)

[artifact-index](#) [artifact-index](#)

這份 artifact index 已經可以作為 Phase 5 製作 PPT、demo、影片與 final report 的素材地圖。

## C. smoke-test-report.md 已完成 real-ZMQ hardening

新版 smoke report 明確寫：

- `HAS_NS3GYM=True`
- `ZMQ mode=real`
- `real-ZMQ enforcement=enabled`
- `allow_dummy=False by default`
- S1 / S2 smoke test both PASS
- throughput non-zero check passed
- dummy fallback 只允許 explicit debug / unit testing with `--allow-dummy`

這代表我前次指出的「dummy fallback 也可能誤通過」風險，這次已在正式報告層補齊。

[smoke-test-report](#)

## D. Excellent report 已明確 PASS / GO

phase4-excellent-acceptance-report.md 最後明確寫：

**Phase 4 Excellent Acceptance: PASS**  
**Recommendation for Phase 5 Final Reporting / Demo / PPT Package: GO**

並說所有治理與文件同步問題已解決，等待 Spec Owner review 後進入 Phase 5 / Change 05。

phase4-excellent-acceptance-rep...

## 4. README 還有一個小瑕疵，但不阻塞 Phase 5

README 已經大致同步：Toolchain 顯示 DQN 已訓練 S1+S2；Change roadmap 顯示 Change 04 Phase 4 Complete；Results Summary 也有 S1 / S2 DQN 結果與 honest limitations。

README

README

唯一小瑕疵是 README 的「How to Train DQN」區塊仍主要示範 S1 command，status 文字也只寫 S1 trained。

README

但同一份 README 後面已明確列出 S2 DQN results、Change 04 complete、S1/S2 trained/eval/compare complete，所以我判定這是**非阻塞型文件精修**，可以放到 Change 05 一併整理，不需要再卡 Phase 4。

## 5. 是否能推進 Phase 5？

可以。

正式判定：

**Phase 4 Excellent Acceptance: PASS**  
**Phase 5 Final Reporting / Demo / PPT Package: GO**

但 Phase 5 的名稱與工作範圍應該修正為：

**Phase 5: Final Reporting / Demo / PPT Package**

不要叫「Phase 5：做 DQN MVP」。

DQN MVP 現在已經做完，Phase 5 是把成果轉成期末交付物。

## 6. Phase 5 前是否要先做 OpenSpec Change 05？

要，而且我強烈建議一定要做。

原因是：OpenSpec 官方說它的目的就是讓人與 AI coding assistant 在寫 code 前先對齊 specs；每個 change 會有自己的 proposal、specs、design、tasks，並讓 reviewer 能理解 change 本身，而不是只看 code。官方 README 也示範 `/opsx:propose` 會建立 `openspec/changes/<change>/`，內含 `proposal.md`、`specs/`、`design.md`、`tasks.md`；Quick Start 也強調先 `openspec init`，再用 `/opsx:propose <what-you-want-to-build>`。  [GitHub](#)

所以 Phase 5 不應該直接叫 Antigravity 做 PPT、報告、demo。  
正確順序應該是：

Step 1：建立 OpenSpec Change 05：reporting-figures-and-demo  
Step 2：你驗收 Change 05 spec  
Step 3：Antigravity 依 Change 05 實作 final deliverables  
Step 4：你驗收 final report / PPT / demo / video script

## 7. Change 05 應該定義什麼？

我建議 Change 05 名稱為：

**reporting-figures-and-demo**

它應該只管 **final deliverables**，不要再改 Phase 4 的 DQN 實驗核心。

Change 05 應包含：

Change 05 項目	內容
<b>proposal.md</b>	為什麼需要把 Phase 3 / Phase 4 成果整理成 final report、PPT、demo、video package
<b>design.md</b>	final narrative、圖表選擇、demo flow、10 分鐘影片結構
<b>tasks.md</b>	README final cleanup、figure selection、PPT outline、demo script、video script、final report
<b>specs/reporting.md</b>	報告章節、資料來源、不得過度宣稱
<b>specs/figures.md</b>	哪些圖進 PPT、哪些只留 appendix
<b>specs/demo.md</b>	demo 要跑哪些 command、展示哪些 artifacts
<b>specs/video-script.md</b>	10 分鐘影片段落與講稿方向
<b>specs/honest-claims.md</b>	DQN 成果如何講，哪些不能講
<b>specs/final-acceptance.md</b>	最後交付物驗收條件

Change 05 必須明確禁止：

- 不再重訓 DQN；
- 不改實驗數據；
- 不修飾數據成「DQN 最強」；
- 不引入 PPO；
- 不新增 IPFS / QUIC 實作；

- 不刪除 BBR / NewReno / CUBIC 對照；
- 不把 reward curve 當作唯一成功證據；
- 不把 S1/S2 結果誇大成 general Internet deployment。

## 8. 我的建議下一步

你現在可以進入下一步，但我建議這樣命名：

**Phase 5 前置：OpenSpec Change 05：reporting-figures-and-demo 規格定義**

不是直接進：

**Phase 5：做 DQN MVP**

DQN MVP 已經完成。現在要做的是期末交付物工程。

我的最終建議：

**Phase 4: PASS**  
**Change 05 should be created before Phase 5 implementation: YES**  
**Phase 5 GO condition: Create and approve Change 05 first**

接下來可以先請我產出 **OpenSpec Change 05：reporting-figures-and-demo** 的正式最佳 Prompt。